

Goal: Build a strong LLM + GenAI foundation in 7 days with real tools like HuggingFace, Ollama, Langchain, and 1 Node.js project.

HELLO WORLD YouTube Channel



Please Subscribe to this channel: [Hello World](#)



Follow me on LinkedIn: [Prince Kumar](#)

● Day 1 – What is LLM? Why LLMs Matter in 2025?

- Famous LLMs: GPT, Mistral, LLaMA, Claude
- Use cases in real life (chatbots, summarizers, auto-coding)
- How does an LLM actually work (basic explanation – tokenization, attention)
- Tools: No tools today – Just basic understanding
- PDF Notes: Terminology cheat-sheet

● Day 2 – HuggingFace 101 & Open-Source LLMs

- What is HuggingFace? Model hub + datasets + inference API
- How to find & test a model from 🤖
- What is the Transformers library? Use pretrained model to generate text
- Hands-on: Run text generation with transformers on Colab
- Tools: HuggingFace Hub, Transformers lib (Colab)
- Bonus: HF Model Picker Cheatsheet

● Day 3 – Ollama + LM Studio: Run LLMs on Your Laptop

- What is Ollama? Why it's revolutionizing local LLMs
- Install Ollama on Windows/Mac
- Try Mistral, LLaMA 3 locally
- LM Studio overview – GUI for LLMs
- Tools: Ollama, LM Studio
- Bonus: List of lightweight models for slow laptops

● Day 4 – Prompt Engineering in Hindi (With Examples)

- What is Prompt Engineering?
- Prompt structure: System | Instruction | Examples
- Roleplay Prompting | Chain-of-thought | Few-shot prompting
- Create a Resume Builder or Email Generator Prompt
- Tools: Ollama / ChatGPT
- Bonus: 10 Copy-Paste Prompts (for devs/students)

● Day 5 – LangChain Basics

- What is LangChain?
- Build a basic chain (LLM + Prompt + Output Parser)
- LangChain Expression Language (LCEL) & `.pipe()` method
- Tools: Node.js + LangchainJS
- Different tools LangChain offers
- Bonus: LangChain concepts mindmap

● Day 6 – LangChain Agents, Tools & Memory

- What are Agents?
- Tools agents can use (search, calculator, APIs, etc.)
- Memory in LangChain (context persistence)
- Code: Make a basic LangChain “Q&A bot” using agents and memory

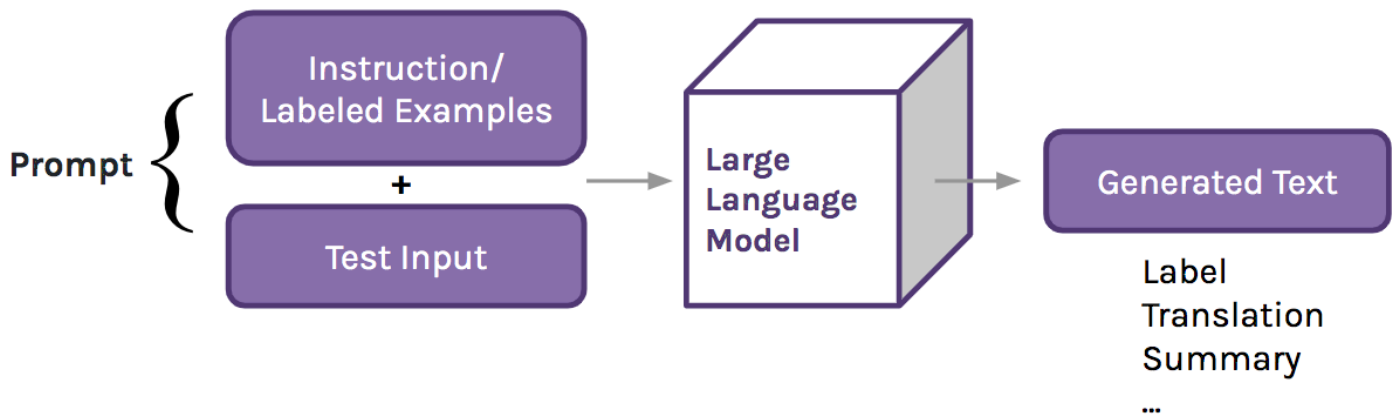
● Day 7 – Project: Create Your Own AI Chatbot using Node.js + Gemini

- Setup project with [Express.js](#)
- Connect Ollama (local) or Google Gemini with the API
- Create a prompt structure
- Build frontend UI with HTML/CSS/JS
- Final result: Talk to LLM like ChatGPT
- Tools: Node.js, Express, Gemini, Langchain
- Bonus: Project Source Code + Flowchart PDF

Day 1: What is LLM? Why LLMs Matter in 2025?

What is LLM?

- **LLM** = *Large Language Model*
- It's a type of **Artificial Intelligence** that can **understand**, **generate**, and **interact using human language**
- Trained on **massive text data** like books, websites, Wikipedia, etc.
- Example: ChatGPT, Claude, Gemini



🧠 Think of LLM as a *very advanced auto-complete engine*, but for full conversations, code, essays, and more.

Why LLMs Matter in 2025?

- Replacing traditional software in many areas
- Powering apps like chatbots, code assistants, AI tutors, and even business analysts
- Easy to integrate in your own apps
- Future of jobs = You + AI (LLMs will be your assistant)

Famous LLMs You Should Know

- **GPT** (OpenAI) – ChatGPT, GPT-4o
- **Claude** (Anthropic - *AI safety and research company*)
- **LLaMA** (Meta, open-source)
- **Mistral** (Open-weight French LLM)
- **Gemini** (Google's LLM)

🧠 Each LLM has different strengths: speed, accuracy, openness, and multilingual abilities.

Real-World Use Cases of LLMs

- **Chatbots**: Like, Helpdesk bots
- **Summarizers**: News apps, research assistants
- **Translation**: Convert Hindi ↔ English easily
- **Auto-coding**: GitHub Copilot, AI code reviewers
- **Tutoring**: AI Teachers like Khanmigo

⚡ Start thinking of how you can build AI tools using LLMs, even if you're a beginner.

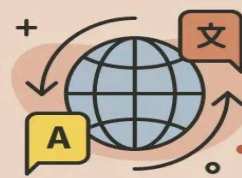
6 Large Language Model Use Cases



Chatbots



Text generation



Language translation



Sentiment analysis



Question answering



Code generation

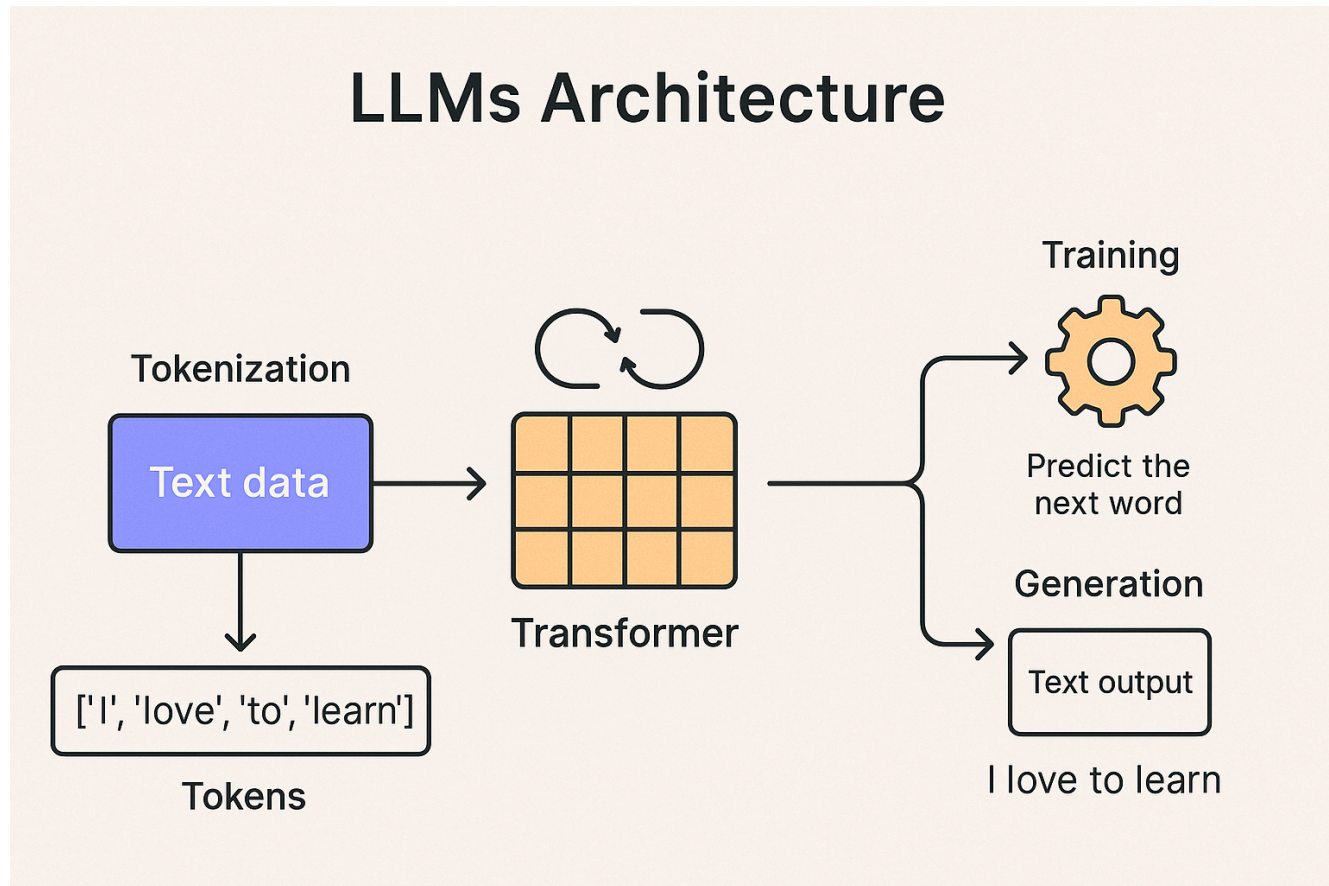
smith.ai

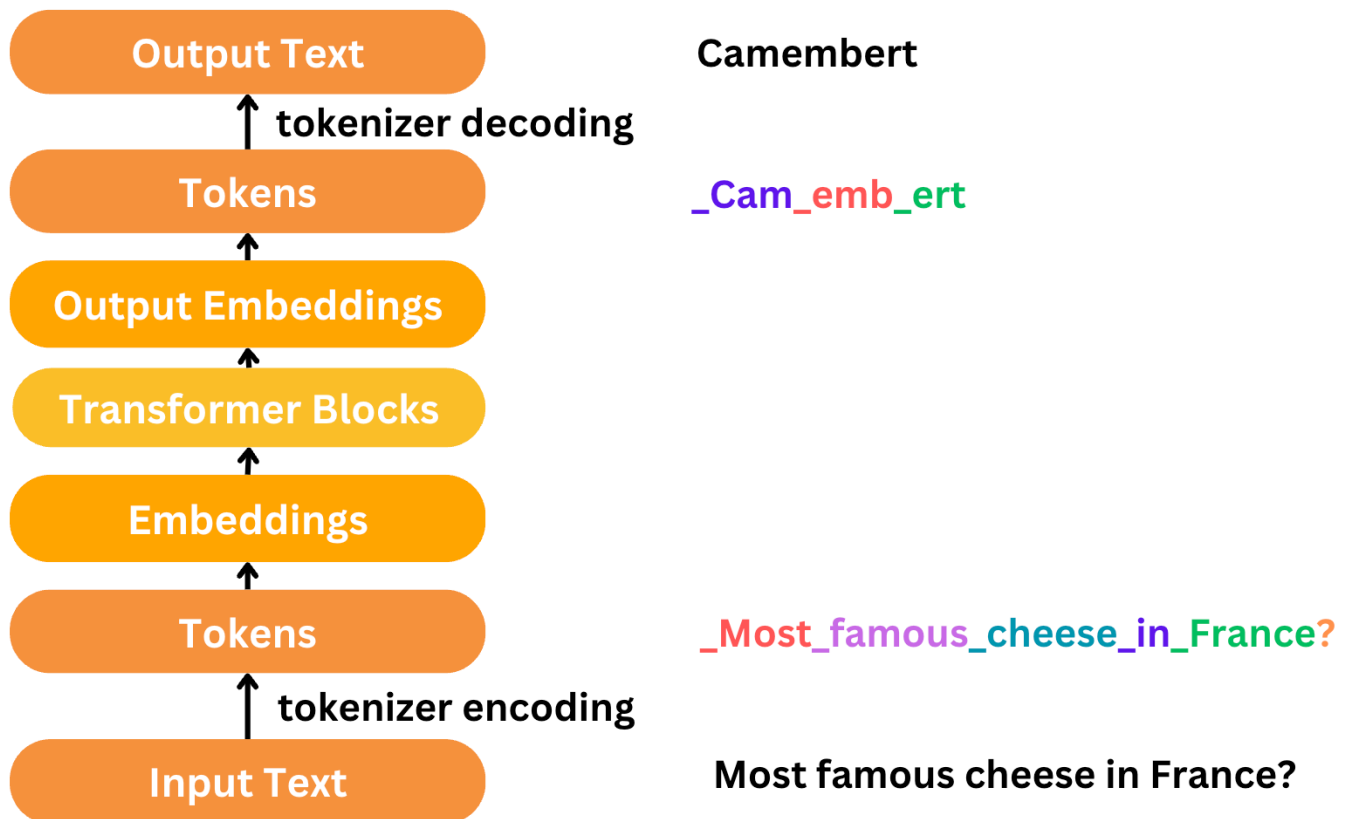
How Does an LLM Actually Work?

🧩 Simple explanation:

1. **Tokenization**: Breaks your text into smaller units (tokens)
2. **Embedding**: Converts tokens into numbers (vectors)
3. **Transformer**: Uses an **attention mechanism** to decide which words matter
4. **Prediction**: Generates the next token (word/letter) based on training

🧠 “Attention is all you need” – the core of how LLMs work





🧩 Step 1: Tokenization

- ◆ This step breaks the input text into smaller units called **tokens**. These tokens could be words, subwords, or even characters (depending on the tokenizer).

Input = "Most famous cheese in France?"

📌 Tokens (subword-style): ["Most", " famous", " cheese", " in", " France", "?"]

These are not just words — spaces matter (see " famous").
This helps the model handle unknown words and multiple languages.

You can also visualise here: <https://platform.openai.com/tokenizer>

🧩 Step 2: Embedding

- ◆ Now, each token is converted into a **vector** – a list of numbers. These vectors **represent the meaning** of the tokens in mathematical space.

"cheese" → [0.12, -0.55, 1.03, ...] (just an example vector)

If a sentence is tokenized into **5 tokens**, you will get **5 separate vectors** — one for each token.

["I", "love", "AI", "model", "s"]

```
[  
  [0.01, -0.92, 0.11, ..., 0.34], // vector for "I"  
  [0.45, 0.82, 0.66, ..., 0.91], // vector for "love"  
  [0.12, 0.34, -0.75, ..., 0.01], // vector for "AI"  
  ...  
]
```

Each **token** → **one vector** (usually 768 or 1024 dimensions depending on the model).

🧠 Do all LLMs give the same vector for **apple**?

No – embeddings vary depending on:

1. **The model** (GPT, LLaMA, BERT all have different embeddings).
2. **The training data** (different LLMs "understand" words differently).
3. **The tokenizer used** (byte-pair encoding, WordPiece, etc.).

🧩 Step 3: Transformer (Attention)

◆ Here comes the smart part: the **transformer** looks at the **entire sentence** and tries to understand:

- What is important?
- What words are related?
- What to focus on?

📌 Example:

- The model learns that "**cheese**" and "**France**" are related.
- "**famous**" adds extra weight.
- The attention mechanism focuses more on "**cheese**" and "**France**".

📌 *This is like a human thinking:*

"Hmm... I've heard of Brie, Camembert... those are cheeses from France."

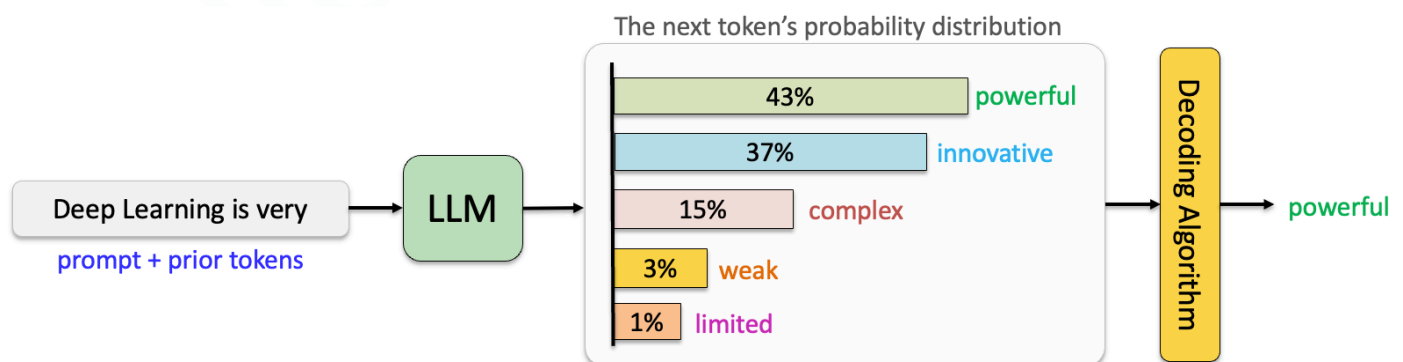
🧩 Step 4: Prediction (Output)

◆ Now the LLM is ready to **predict the next most likely tokens**.

📌 Given the input: "Most famous cheese in France?"

The model might generate: "Brie"

Or, if allowed to continue: "Brie is one of the most well-known cheeses in France."



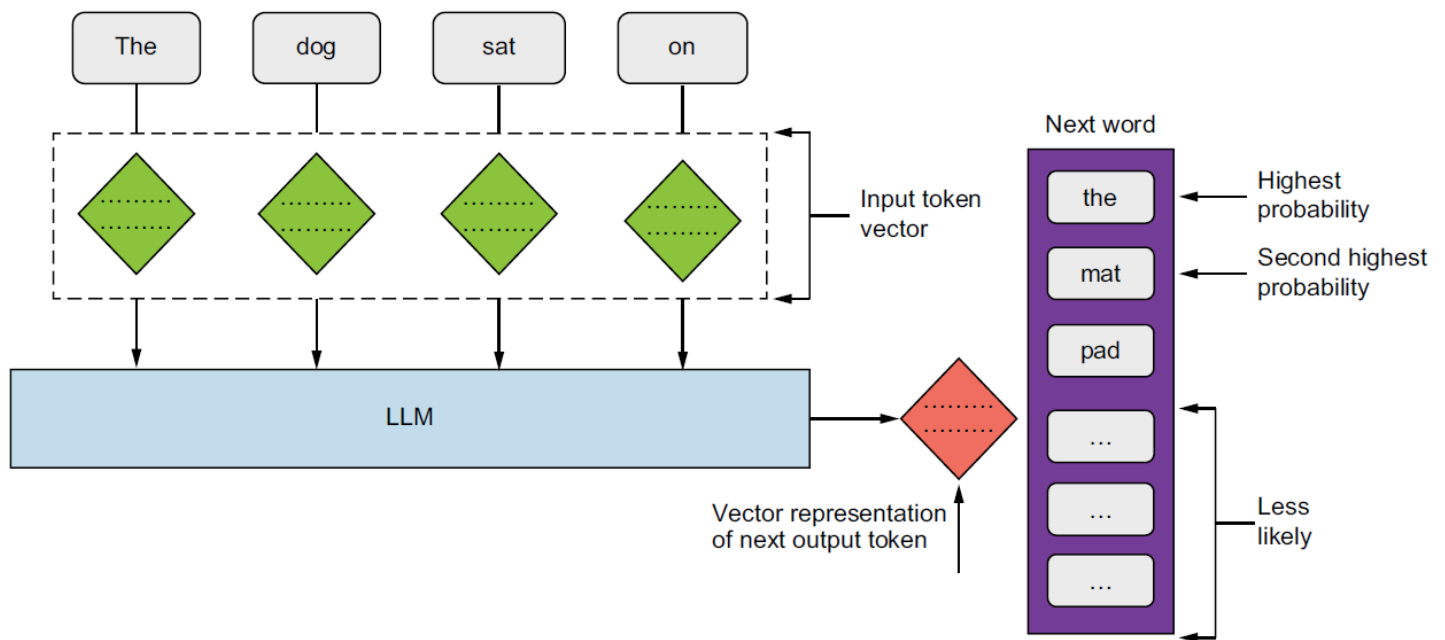
👉 The prediction depends on:

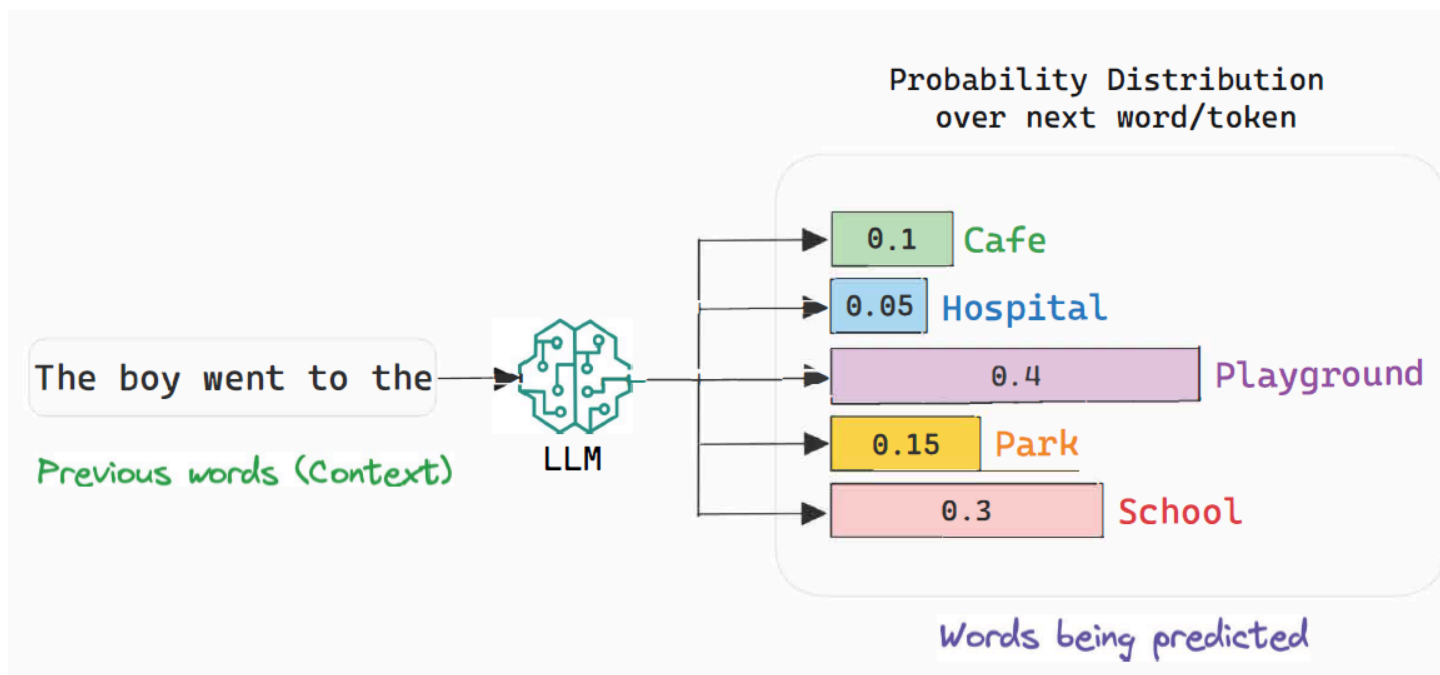
- The training data
- The prompt
- Temperature (creativity)
- Max token length

🧠 Tip for Students:

LLM doesn't "search" the web — it “predicts” based on what it has already **learned** from massive training data.

LLM – Next token predictor

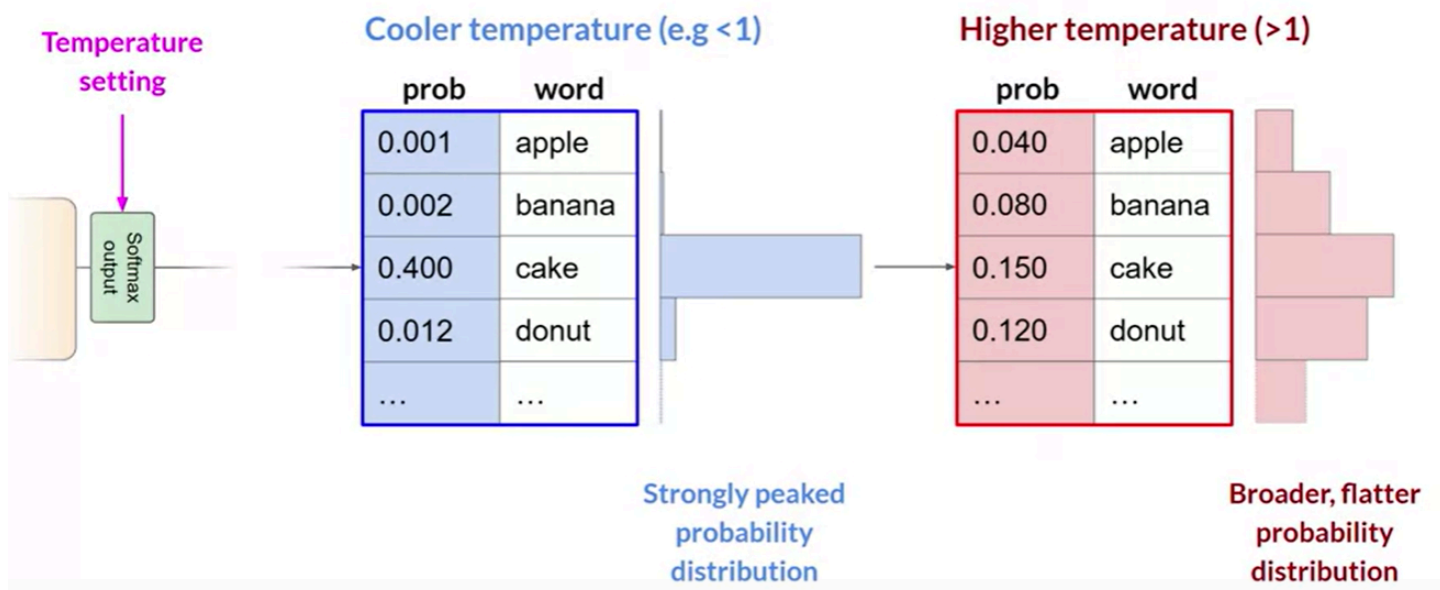


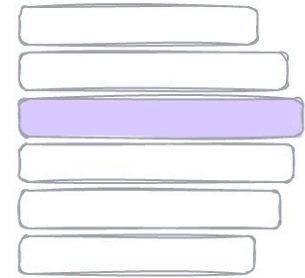
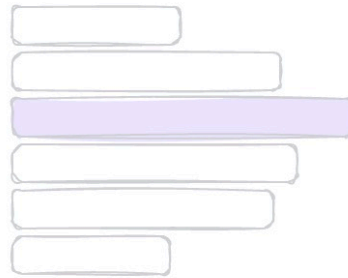
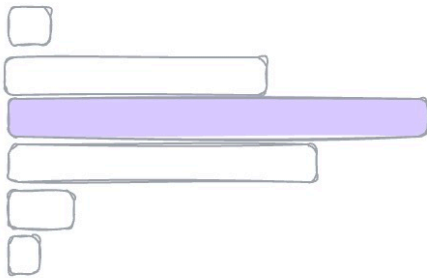


The **token with the highest probability** is **not always chosen**, and that's because of **sampling strategies and decoding settings** like:

🔥 1. Temperature

- Controls **randomness**.
- Lower (~ 0.2): **More deterministic** (almost always picks top token).
- Higher ($\sim 1.0+$): **More diverse**, samples from wider range.





low temperature

high temperature

2. Top-k Sampling

- Only consider the **top-k most probable tokens** (e.g., top 40).
- Randomly sample from them based on their probabilities.
- Discards the rest.

3. Top-p Sampling (Nucleus Sampling)

- Picks the **smallest set of tokens** whose cumulative probability is $\geq p$ (e.g., 0.9).
- Then samples **within that set**.
- More adaptive than top-k.

Top-k sampling (set k to 2)

The dog is

[Top-k]	[Token]	[Top-p (p: Probability)]
Top-1	very	0.81
Top-2	good	0.14
Top-3	bad	0.02
Top-4	wrong	0.002
...	...	...
Top-4899	was	0.0001
Top-4900	will	0.0001
...	...	...

(Sampling list)

Top-p sampling (set p to 0.96)

The dog is

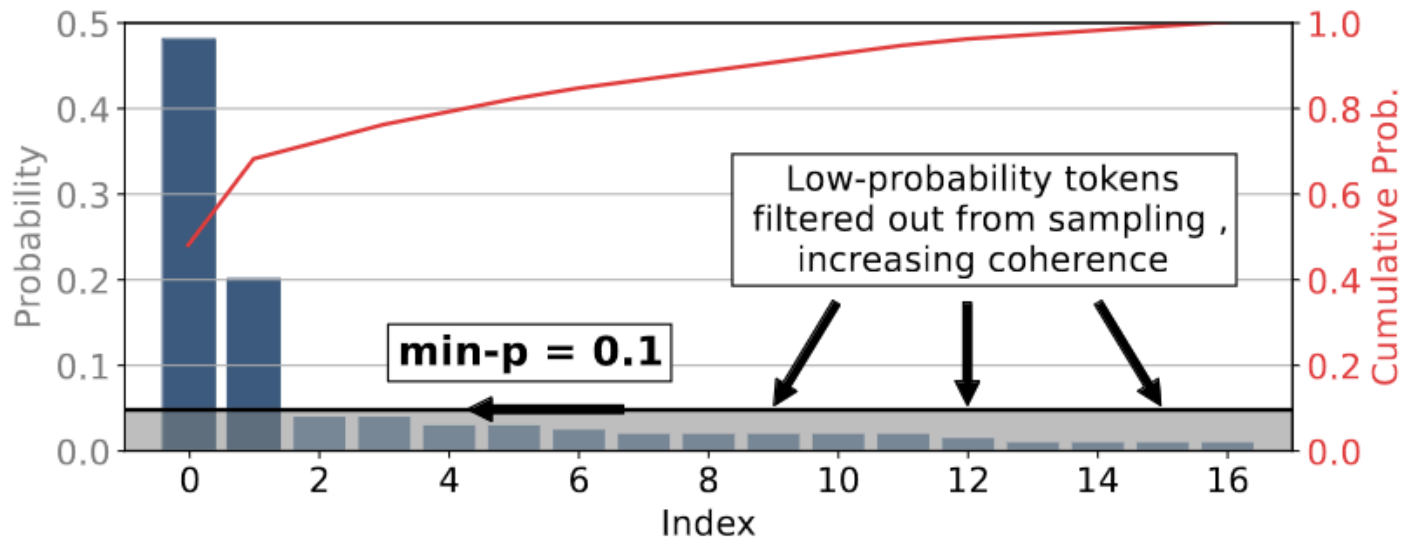
[Top-k]	[Token]	[Top-p (p: Probability)]
Top-1	very	0.81
Top-2	good	0.14
Top-3	bad	0.02
Top-4	wrong	0.002
...	...	...
Top-4899	was	0.0001
Top-4900	will	0.0001
...	...	...

(Sampling list)

Cumulative probability exceeds 0.96
(0.81+0.14+0.02 > 0.96)

4. Min-p (minimum probability cutoff)

- Discards tokens whose probability is **below a threshold** (e.g., 0.1).
- Helps avoid unlikely garbage outputs.



7 LLM Generation Parameters To Know

		Explanation	Value Range
max_tokens		Upper limit for the number of tokens the model generates	1 to infinity
temperature		Controls randomness in output. A higher temperature makes more creative and diverse	0 to 2 (Common Range)
top_p		Controls probability distribution is considered when sampling tokens	0 to 1
top_k		Limits the number of top probable tokens to sample from	1 to infinity
frequency penalty		Penalizes token repetition based on frequency. Positive values reduce repetition	-2 to 2
presence penalty		Encourages the model to use new tokens that haven't been generated	-2 to 2
stop		A list of tokens where the model will stop generating further tokens	Custom List

Learning Resource: <https://www.nb-data.com/p/7-llm-generation-parameters-to-know>

Day 2: HuggingFace & Open-Source LLMs

Today, you'll explore the **HuggingFace** ecosystem, learn how to search and use open-source LLMs, understand quantized models, and run text generation in Colab using the 🧠 transformers library.

What is HuggingFace?

A platform for hosting ML models, datasets, and tools.

Like GitHub, but for AI models.

Running AI applications directly from the browser via **Spaces**

🔗 Website: <https://huggingface.co>

HuggingFace Hub includes:

- **Models:** LLMs like GPT2, Mistral, LLaMA, etc.
- **Datasets:** IMDB, SQuAD, etc
- **Spaces:** Web apps built using models.
- **Inference API:** Test models directly in the browser or through the API.

Component	Description
 Models	Pretrained models like GPT2, Mistral, LLaMA, BERT
 Datasets	Public datasets like SQuAD, IMDB, and Common Crawl
 Inference API	Free testing interface for models
 Spaces	Public web apps built using HuggingFace models

In simple words:

Inference = using a trained AI model to make predictions or generate results.

What Is Hugging Face Inference API?

The Inference API by Hugging Face lets you use AI models hosted on Hugging Face servers, without downloading or running them locally.

So you don't need a GPU or set up PyTorch or anything.

What Happens Behind the Scenes?

1. You choose a model (e.g., `gpt2`, `facebook/bart-large`, `mistralai/Mistral-7B`)
2. Hugging Face hosts that model on their own powerful GPUs
3. You make a REST API call to `https://api-inference.huggingface.co/models/{model-name}`
4. The model **runs inference** = takes your input, generates prediction/output
5. You get the result back

Why Do People Always Say "You Need a GPU for AI"?

Because **GPUs are built for doing many math calculations at the same time** — which is exactly what AI models need.

What is a CPU?

- **CPU = Central Processing Unit**
- It's like your computer's **main brain**
- It's designed to handle **a few tasks very fast**
- Good at: Opening apps, Running OS, Doing a few calculations at a time

A **very smart person**, doing 1 math problem at a time — super accurate, but only one thing at once.



What is a GPU?

- **GPU = Graphics Processing Unit**
- Originally made for games, to draw images (lots of pixels, colors)
- But later we realized: “**Wow, it can do 1000s of math problems in parallel!**”



It has **thousands of small cores** that can do:

- Matrix multiplications
- Vector operations
- Math needed for AI

[A **core** is like a **mini-brain** inside your computer's processor. Each core can **do one task (like a math problem) at a time.**]



In a CPU

- Most modern CPUs have **4 to 16 cores**
- So your computer can run **4 to 16 tasks at once** (parallel processing)



Imagine a **CPU with 4 cores** like:

4 students solving 4 different problems at the same time.



In a GPU

GPUs have **hundreds or even thousands of smaller cores**

(e.g., *NVIDIA RTX 4090 has 16,384 cores*)



Imagine a GPU with 1000 cores like:

A classroom with **1000 students**, each doing a tiny piece of a huge puzzle — all at once. Because AI tasks (like LLMs, image generation, etc.) involve:

- Multiplying giant matrices (like 1000x1000 numbers)
- Repeating this **millions of times**

The more cores you have:

The more pieces of the task can be done **simultaneously**

So the job finishes **much faster**



✓ Can You Run AI Models on a CPU?

You **can run small AI models on a CPU**, especially **low-size models**, and it's a great way to get started without needing a GPU.

Small LLMs like `distilBERT`, `TinyLlama`, `GPT2-small`, `mistral-7B` (with quantization)

Tasks like:

- Text classification
- Question-answering
- Basic text generation
- Sentiment analysis

🧠 These are lightweight models that don't need thousands of GPU cores.

♦ What is a Quantized Model?

Quantization means reducing the model size by storing numbers with lower precision (e.g. int4 instead of float32), making them:

- 🟢 Faster to load
- 🟢 Lighter in size (less storage required)
- 🟡 Slightly less accurate (minimal drop)

🎯 1. What Do You Mean by "Model Parameters"?

In AI models, **parameters** are the numbers the model "learns" during training.

AI Model = Predict Next word

For that, it needs data

Imagine:

- You train a model to translate Hindi to English.
- It learns millions (or billions) of **rules, weights, and values**.
- These are all **stored as numbers**.

🧠 For example:

- GPT-2 has **124 million parameters**
- GPT-3 has **175 billion parameters**

A "parameter" = A learned number (like a weight) the model uses to generate predictions.
A **parameter** is always a **number (value)** — usually a **weight**— stored inside the model.

📄 Summary Table

Term	Simple Meaning	Example
Weight	How important something is	"great" = 0.95, "boring" = -0.6
Value	The actual number stored in the model	0.95, -0.6, etc.
Rule	A learned pattern or decision that the model makes	"If not before, good = negative."

🤖 AI doesn't write if-else — it **learns** patterns by adjusting weights from lots of data.

Final Analogy

Imagine you're learning to drive:

- You **learn** how much to press the accelerator (weight)
- You **store** that info in your brain (value)
- After many drives, you know what to do at a turn (rule)

AI does the same, but in **mathematical form**.

1234 2. How to Calculate **Model Size** from Parameters?

Each parameter is a number.

Usually stored as:

- `float32` → takes **4 bytes** (default)
- `int8` → takes **1 byte**
- `int4` → takes **0.5 byte**

Memory unit	Description
Kilo Byte	1 KB = 1024 Bytes
Mega Byte	1 MB = 1024 KB
Giga Byte	1 GB = 1024 MB
Tera Byte	1 TB = 1024 GB
Peta Byte	1 PB = 1024 TB
Hexa Byte	1 EB = 1024 PB
Zetta Byte	1 ZB = 1024 EB
Yotta Byte	1 YB = 1024 ZB
Bronto Byte	1 Bronto Byte = 1024 YB
Geop Byte	1 Geo Byte = 1024 Bronto Bytes

Formula:

Model Size = Number of Parameters × Size of each parameter

Example:

Model	Parameters	Precision	Size
GPT-2	124M	float32 (4 bytes)	$124M \times 4 = \sim 496 \text{ MB}$
GPT-2 (int8)	124M	int8 (1 byte)	$124M \times 1 = \sim 124 \text{ MB}$
LLaMA 7B	7B	float32	$7B \times 4 = \sim 28 \text{ GB}$
LLaMA 7B (4-bit)	7B	int4 (0.5 byte)	$7B \times 0.5 = \sim 3.5 \text{ GB}$

So when people say “**quantized model is smaller**”, it’s because we **store those parameters with lower precision**.

Quantization means converting the model’s numbers (parameters) from high-precision to low-precision to make the model smaller and faster.

Analogy:

Imagine storing ₹123.456789

Full: float32 (full precision) → takes more space

Rounded: int4 or int8 → less accurate, but way smaller

Types of Precision:

Type	Bits	Bytes	Accuracy	Size
float32	32 bits	4 bytes	🔥 High	❌ Big
int8	8 bits	1 byte	👍 Good	✅ Smaller
int4	4 bits	0.5 byte	👉 Acceptable	✅✅ Smallest

✅ What Is a **Low-Size Model**?

A **low-size model**:

- Has **fewer parameters** (e.g., 60M, 124M)
- Or is **quantized** (4-bit, 8-bit)
- Can run on **CPU or less RAM** (under 2–4 GB)

Naming Convention of Models

Model names often include **architecture, size, quantization, and version**:

Example: TheBloke/Mistral-7B-Instruct-v0.1-GGUF

Part	Meaning
TheBloke	Organization/maintainer
Mistral	Base model architecture
7B	Model size (7 billion parameters)
Instruct-v0.1	Instruction-tuned version
GGUF	File format used in quantized models



What Does 7B, 13B, 70B Mean?

These represent the **number of parameters** in the model:

Model Size	Description	Requirements
1B–7B	Lightweight models	Can run locally
13B–30B	Mid-size, better reasoning	Requires GPU
65B–180B	Powerful but slow and heavy	Needs high-end GPU or cloud



More parameters = more intelligence **but** needs more RAM and time

Bonus: HuggingFace Model Picker Cheatsheet

Use Case	Model Example	Notes
Chatbot	mistralai/Mistral-7B-Instruct	Fast, instruction-tuned
Summarization	facebook/bart-large-cnn	Great for articles
Code Generation	Salesforce/codegen-2B	For Python/JS code
Hindi LLMs	ai4bharat/IndicBART	Multilingual support
Quantized Model	TheBloke/Mistral-7B-GGUF	Lightweight, fast

Example: https://huggingface.co/google/gemma-3-4b-it-qat-q4_0-gguf



[Hello World](#)

What is 🤗 Transformers Library?

- **Official Python library by HuggingFace.**
- Gives access to hundreds of pretrained models.
- Just 3–4 lines of code to load and use LLMs.
- Supports multiple tasks: text generation, classification, translation, etc.

🔧 Hands-On Activity

Goal: Generate text using a pretrained model

1. Open Google Colab
2. Paste and Run:

```
!pip install transformers

from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")
output = generator("Bharat ka AI future", max_length=50,
num_return_sequences=1)

print(output[0]["generated_text"])
```

✅ You'll see GPT-2 generate Hindi+English text based on the prompt.

✅ **max_length** (int)

This defines the **total length** of the generated sequence **including the input prompt tokens**.

📌 It means: “how long should the final output be (including input)?”

- If the prompt has 10 tokens, and **max_length=50**, the model will generate **up to 40 new tokens**.
- This is a **hard limit**. If the model hits it, it stops generating even if the sentence isn't complete.

✓ num_return_sequences (int)

This specifies **how many different completions** you want the model to generate.

📌 It means: “give me this many variations of the output.”

- If set to **1** → you get **one response**
- If set to **3** → you get **three different versions** of the response (helpful for sampling diverse completions)

Day 3: Ollama + LM Studio: Run LLMs on Your Laptop

By the end of today, you'll be able to **run open-source LLMs like Mistral or LLaMA 3 on your laptop** — no cloud, no API key, no GPU required.

What is Ollama?

Ollama is a command-line tool that makes it easy to run **open-source LLMs** (like Mistral, LLaMA 3, etc.) on your own machine.

- A simple CLI tool to run LLMs *locally*.
- Think of it as “**Docker for LLMs**” — download a model once, and use it offline.
- Supports models like **Mistral**, **LLaMA 3**, **Gemma**, **Code LLaMA**, etc.
- Great for privacy, low-latency, and cost savings.

<https://ollama.com/>

How to Install Ollama

On Windows:

- Install WSL (Windows Subsystem for Linux):
<https://docs.microsoft.com/en-us/windows/wsl/install>

WSL, or **Windows Subsystem for Linux**, is a feature of Windows that allows users to run a Linux environment directly on Windows without the need for a virtual machine or dual-booting

Then install Ollama:

```
curl -fsSL https://ollama.com/install.sh | sh
```

On Mac:

- Run this command in Terminal:

```
curl -fsSL https://ollama.com/install.sh | sh
```

After install, type `ollama run mistral` to try your first model!

How Ollama Works (Behind the Scenes)

1. **Where does it store models?**

By default, Ollama stores downloaded models on your local disk:

```
~/.ollama/models
```

You can explore them here like cached files.

2. **Where does it get models from?**

Ollama fetches models from the **official Ollama model registry**:

- <https://ollama.com/library>
- These models are pre-converted to an efficient format (quantized **.gguf** under the hood).

You **don't need to visit HuggingFace manually** unless you're doing advanced stuff.

Step-by-Step: Using Ollama

✓ Step 1: Install Ollama

✓ Step 2: Pull a Model

```
ollama pull mistral
```

This downloads the **Mistral 7B model** to your local disk.

✓ Step 3: Run the Model

```
ollama run mistral
```

This launches an interactive prompt (like ChatGPT in terminal). Type any question, e.g.:

```
> What is quantum computing?
```

✅ Step 4: Use via API (Text Generation in Node/Python)

By default, Ollama also runs a local **REST API** on:

```
http://localhost:11434
```

👤 Example: Generate Text via [Node.js](#)

```
const axios = require('axios');

const response = await axios.post('http://localhost:11434/api/generate', {
  model: 'mistral',
  prompt: 'Explain Blockchain in simple terms',
});

console.log(response.data.response);
```

You can also use **curl**:

```
curl -X POST http://localhost:11434/api/generate \
-H "Content-Type: application/json" \
-d '{
  "model": "tinyllama",
  "prompt": "Explain quantum computing in simple terms.",
  "stream": false
}'
```

✅ Step 5: List All Installed Models

```
ollama list
```

✅ Step 6: Remove/Unload a Model

To free up space:

```
ollama rm mistral
```

✓ Step 7: Run Custom or Offline Models (Optional)

If you have a `.gguf` file or a model from HuggingFace:

1. Place it inside `~/.ollama/models`.
2. Create a `Modelfile` to define how it runs.
3. Run:

```
ollama create my-model -f Modelfile
ollama run my-model
```

You can also share and publish your own models to others via `ollama push`.

Sample Use Case

```
"How do I build a chatbot using Ollama + Express?"
```

You use `ollama run` in backend or call its **API** (`/generate`) from Node.js server, and build a frontend to send messages to it — like ChatGPT but local.

Summary Cheatsheet

Command	Description
<code>ollama pull model</code>	Download a model
<code>ollama run model</code>	Run the model in terminal
<code>ollama list</code>	Show installed models
<code>ollama rm model</code>	Remove a model
<code>curl localhost:11434/api/...</code>	Access model via HTTP

You may face an error like this:

```
prince@Mac ~ % ollama run tinyllama
```

```
pulling manifest
```

```
Error: max retries exceeded: Get
```

```
"https://dd20bb891979d25aebc8bec07b2b3bbc.r2.cloudflarestorage.com/ollama/docker/registry/v2/blobs/sha256/2a/2af3b81862c6be03c769683af18efdadb2c33"
```

Then add

DNS Record in your system: 1.1.1.1

```
prince@Mac ~ % ollama run tinyllama
pulling manifest
pulling manifest
pulling manifest
pulling manifest
pulling manifest
pulling manifest
pulling 2af3b81862c6: 74%  | 473 MB/637 MB 3.6 MB/s 45s
```

Feature	Hugging Face	Ollama
What it is	A massive open-source model hub for ML/LLMs	A local LLM runtime focused on running models on your own machine
Where models run	Mostly cloud-based or via Colab/local Python	Entirely local (on your laptop/desktop)
Model format	Transformers (PyTorch, TensorFlow, Safetensors, etc.)	Ollama's own format (GGUF under the hood)
Interface	Python-based (transformers , diffusers , etc.)	CLI + REST API (ollama run , /api/generate)
Hosting	Hosts thousands of models by researchers and orgs	Downloads from curated model registry (Ollama Hub)
Usage	Best for research, training, and online inference	Best for <i>private</i> , <i>offline</i> inference
Models Available	GPT2, GPT-J, LLaMA, Mistral, BERT, etc.	Mistral, LLaMA2/3, TinyLLaMA, Codellama, Phi, etc. (quantized)
Ease of Use	Requires Python setup and environment	One-line install, models ready with ollama run

Customization	Easily fine-tune and upload custom models	Custom model YAML configs supported but limited fine-tuning
----------------------	---	---

Summary:

- **Hugging Face** = *A global library of models for devs and researchers*. Best when you need access to **any model**, large-scale deployment, or training.
- **Ollama** = *Fast, private, local tool to run quantized LLMs easily on your laptop*, with no setup hassles or API costs.

Try These Models Locally

Model Name	Parameters	Use Case	Command
mistral	7B	General Purpose	<code>ollama run mistral</code>
llama3	8B	Chat Assistant	<code>ollama run llama3</code>
gemma	2B	Lightweight AI	<code>ollama run gemma</code>
codellama:7b	7B	Code Generation	<code>ollama run codellama</code>
tinylama	1B	Ultra Lightweight	<code>ollama run tinylama</code>

🧠 What is LM Studio?

LM Studio is a free desktop app that lets you **run large language models (LLMs) offline**, using a simple chat interface – **no coding or terminal required**.

It's like running ChatGPT or Claude *locally* on your laptop.

How LM Studio Works Internally

-  **Model Format Supported:** Only **GGUF models** (quantized format) — same as what Ollama and llama.cpp use.
-  **Model Loading:** It downloads and stores models *locally on your computer* (typically in a folder like `~/ .cache` or a user-defined path).
-  **Inference Engine:** Internally, it uses optimized backends like `llama.cpp`

🔍 Where to Find Models for LM Studio?

LM Studio provides a **built-in model search** powered by Hugging Face.

1. Open LM Studio app
2. Go to “**Models**” tab
3. Type model name like `Mistral`, `LLaMA`, `Phi`, `TinyLLaMA`, etc.
4. You'll see only **GGUF-supported models**
5. Click **Download** and wait — the model is now stored locally

Step 1: Install LM Studio

- Download from <https://lmstudio.ai>
- Install on Windows/macOS/Linux

Step 2: Search and Download a Model

- Open LM Studio
- Go to **Models** → **HuggingFace Search**
- Pick a model like:
 - **Mistral-7B-Instruct-GGUF**
 - **TinyLLaMA-1.1B-GGUF**
- Click **Download**

Step 3: Load the Model

- Once downloaded, go to **Chat** tab
- Select the model from the dropdown
- Wait a few seconds — it will load into RAM

Step 4: Chat with the Model

- Type your prompt in the chat box
- Hit Enter — get response locally (no internet needed!)



Unload / Remove Model

- Go to **Settings** → **Model Storage**
- You can:
 - Unload models from memory (free RAM)
 - Delete downloaded models from disk

Use Case: Text Generation

- You can use LM Studio for:
 -  Brainstorming ideas
 -  Studying concepts
 -  Local Chatbots
 -  Secure private chat without internet

Bonus Settings You Can Play With

- **Temperature:** Controls creativity
- **Top_p, Top_k:** Controls randomness
- **Context Length:** How many tokens model can remember
- **Prompt Templates:** For chat-style prompt formatting (like ChatGPT)

What is GGUF Format?

- Optimized model format used in **LM Studio**, **Ollama**, and **Text Generation Web UI**.
- Supports quantized models (smaller size, faster on CPU).

Bonus: List of Lightweight Models for Low-End Laptops

Model Name	Size (GGUF)	Description
<code>tinyllama</code>	~0.5GB	Very small, good for testing
<code>gemma:2b</code>	~1.5GB	Google's open-source model
<code>mistral:instruct</code>	~4GB	Great for QnA and chat tasks
<code>llama3:8b</code>	~7GB	Meta's latest LLM (8B params)

Pro Tips

- Use `ollama pull mistral` to download a model without running it.
- In LM Studio, you can download models directly by entering the HuggingFace link.
- Check CPU/RAM usage while running models to monitor performance.

Day 4: Prompt Engineering in Hindi (With Examples)

Make students **confident** in writing effective prompts that **guide any LLM (like ChatGPT/Ollama)** to generate better, more structured, and more useful responses.

What is Prompt Engineering?

- It is the **art and science of asking the right question** to an LLM.
- LLMs respond based on your input, so better prompts = better answers.
- Like talking to a very smart assistant. Your **clarity = their quality**.

♦ Real-world analogy:

Poor prompt: “Tell me about India.”

Good prompt: “Act like a travel guide and explain the top 3 places to visit in India for a solo traveler in 2 days.”

2. Prompt Structure Basics

Most modern prompts follow this pattern:

```
[System Prompt] (who the AI is)
+ [Instruction] (what you want it to do)
+ [Input/Examples] (how you want it done)
```

♦ Example:

```
System: You are a helpful career coach.
Instruction: Create a professional LinkedIn bio.
Input: Name - Prince, Role - Backend Developer, Experience - 3 years
```

3. Types of Prompting Techniques

Teach 3 main types of prompting (with examples):

✓ a) Roleplay Prompting

You're telling the AI to "pretend" to be someone specific — like a teacher, co-founder, doctor, or even a 5-year-old. This changes the **tone, context, and perspective** of the answer.

“Act like a startup co-founder and explain AI to a 12-year-old.”

✓ b) Chain of Thought Prompting

You're asking the LLM to **think out loud** and show the reasoning behind each step, instead of jumping to the final answer.

“Explain step-by-step how to prepare for the GATE exam while managing college.”

Helps the LLM break down thoughts logically.

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✓

✓ c) Few-Shot Prompting

Give **2-3 examples**, then ask it to generate a similar one.

Input:

Text: I love coding.

Output: Main coding karna pasand karta hoon.

Text: She **is** a good teacher.

Output: Woh ek achhi teacher hai.

Now translate:

Text: He **is** my best **friend**.

🔧 4. Hands-on Practice

Try these using **Ollama** or **ChatGPT** (free web app):

♦ Prompt: Resume Generator

You are a professional **resume** writer.

Create a **1-page resume** for:

Name: Ankit Singh

Role: Frontend Developer

Skills: HTML, CSS, React, Tailwind

Experience: **1** year internship

♦ Prompt: Email Generator

Write a polite leave request email **to** college professor **for** **3-day** sick leave.

🎁 5. Bonus Resource: 10 Copy-Paste Prompts

Purpose	Prompt
 Learn anything fast	"Explain [topic] to a beginner in Hindi with simple words and 3 real-life examples."
 Resume Generator	"Create a resume for a fresher applying to software engineering jobs."
 Email Drafting	"Write a formal email for an internship application."
 Idea Brainstorming	"Suggest 5 AI project ideas for beginners with Node.js."
 Quiz Generator	"Make 5 MCQs on Data Structures with answers."
 Daily Planner	"Act like a productivity coach and create a 3-hour study plan."
 App Pitch	"Pitch an EdTech app idea in 3 lines to investors."
 Teaching Help	"You are a Python teacher. Explain recursion in Hindi."
 Summary Tool	"Summarize this text in bullet points under 100 words."
 Goal Planning	"Create a weekly goal tracker for DSA preparation."

✓ Here are **5 solid prompt pairs** (Poor vs. Good)

✗ **Poor prompt:** "Explain Recursion."

✓ **Good prompt:** "Act like a CS professor and explain recursion to a 1st-year student using a real-life example in less than 100 words."

✗ **Poor prompt:** "Write my resume."

✓ **Good prompt:** "Act like a tech recruiter. Here's my project: [describe project]. Help me write 2 impactful resume bullet points for this."

✗ **Poor prompt:** "Give me some project ideas."

✓ **Good prompt:** "I'm a 2nd-year CS student with basic web dev skills. Suggest 3 unique project ideas using AI + Node.js that are beginner-friendly and look good in a resume."

✗ **Poor prompt:** "Write an email for internship."

✓ **Good prompt:** “Write a polite and confident email to a startup founder asking for internship opportunities. Mention I’m a final-year student with Node.js skills.”

✗ **Poor prompt:** “How to learn DSA?”

✓ **Good prompt:** “Act like a mentor and make a 30-day DSA learning schedule for someone who has 1 hour daily and wants to prepare for product-based companies.”

Some Important points related to prompt Engineering:-

✓ 1. Improve Accuracy of Responses

- Crafting clear and specific prompts increases the chance of getting correct answers from LLMs.
- Especially helpful when LLMs give incorrect or vague outputs.

✓ 2. Customize Output Format

- You can define the **style, tone, length**, or even the **label format** of the response.
- Example: Ask for lowercase output or summary in 2 lines.

✓ 3. Guide Reasoning & Logic

- With techniques like **Chain-of-Thought (CoT)**, you can make the model **think step-by-step** for complex reasoning or arithmetic problems.

✓ 4. Extract Information from Text

- Use prompts to **extract names, dates, topics, sentiments**, or any structured info from unstructured paragraphs.

✓ 5. Summarize Long Content

- Ask the LLM to **summarize** blogs, articles, or documents in concise formats.
- You can specify summary length, tone (formal/friendly), or style (bullet points, one-liner).

✓ 6. Answer Questions (QA)

- Use **Zero-shot** or **Few-shot prompting** for Q&A tasks.
- Examples help models understand how to format responses more accurately.

✓ 7. Classify Text

- Sentiment analysis, topic classification, spam detection, etc., using prompts that define label formats.

✓ 8. Generate Code

- Prompt LLMs to write or explain code.
- Specify language, format, and whether to include comments.
- Example: *"Write a Python function for binary search with comments."*

✓ 9. Translate or Rephrase Text

- Translate between languages or change tone (e.g., formal to casual).
- Example: *"Translate this to Hindi in a friendly tone."*

✓ 10. Create Role-Based Conversations

- Use **Roleplay Prompting** to simulate conversations with specific personas.
- E.g., "Act like a startup founder explaining AI to a 12-year-old."

✓ 11. Build Smart Assistants or Chatbots

- Prompt engineering powers **intent-based interactions** for customer support, resume builder, study assistant, etc.

✓ 12. Inject Domain Knowledge

- Using **Knowledge Prompting**, you can add relevant facts to improve output accuracy.
- Especially helpful in medical, legal, or educational contexts.

✓ 13. Enhance Model Reliability with Self-Consistency

- Generate **multiple answers** and choose the most frequent one for **reliable** results in reasoning tasks.

✓ 14. Generate Structured Data

- Ask models to return output in JSON, tables, or formatted text for use in downstream apps or automation.

✓ 15. Boost Creativity or Content Writing

- Use prompts for storytelling, blog writing, social media posts, poetry, etc.

✓ 16. Simulate Exams or Interviews

- Generate mock questions, quizzes, or simulate technical/HR interviews.

✓ 17. Debug or Fix Code

- Prompt the model to **identify bugs** or optimize your code snippets.

✓ 18. Personalize Outputs

- Tailor responses based on audience — e.g., explain to a 5th grader, use a sarcastic tone, etc.

Day 5: *LangChain Basics + LLM Agents*

What is LangChain?

LangChain is a **framework** (like React for frontend) that helps you build **LLM-powered applications** using small building blocks like:

- LLMs (like GPT, Mistral)
- Prompts
- Memory
- Tools/APIs
- Chains & Agents

LangChain makes it **easier to connect LLMs with real-world data, tools, and logic** (like calculators, search, APIs, etc.).

Key Concepts of LangChain

a) **LLM**: The brain (GPT, Mistral, Claude, etc.)

b) **Prompt**: What do you ask the LLM (like “Generate a blog post on AI”)

c) **Output Parser**: Helps convert the LLM’s response into a structured format (JSON, number, list, etc.)

d) **Chain**: A sequence like:

→ Prompt → LLM → Parse → Output
(Used when the flow is fixed)

e) **Memory**: Keeps context between interactions (e.g., Chat History)

f) **Agent**: Think of it like a smart decision-maker. It can:

- Decide what tool to use
- Think step-by-step
- Call external tools
- Act like a mini AI worker

There will be confusion between "chains", "agents", and "LLM apps"

You're not alone — "**chains**", "**agents**", and "**LLM apps**" in LangChain *can* be confusing at first. Let me break it down **step by step** with a beginner-friendly explanation and analogy.



🧩 First, the Big Picture:

With LangChain, you are building **AI-powered applications** — these can be:

- Simple: like a chatbot that answers a question
- complex: like an assistant that books flights, searches the web, summarizes articles, and talks to APIs

To build these apps, LangChain gives you two powerful tools:

✅ 1. Chains

✅ 2. Agents

LangChain gives you more than just "Chains" and "Agents".

Chains and Agents are the **two most important “high-level tools”**, but LangChain provides a **full toolbox** to help you build complete AI-powered apps.

📦 What is a Chain?

Think of a **chain** like a **recipe** — a **fixed set of steps** that happen **in order**, every time.

🔄 **Example Analogy:**

Making tea (Chain):

1. Boil water
2. Add tea leaves
3. Pour into cup
4. Serve

You do it **step-by-step**, always in the **same order**.

🔧 **In LangChain:**

A **chain** means connecting building blocks together:

- LLM (like GPT or Gemini)
- Prompt Template (the question or task)
- Output Parser (to format the output)
- Optional: Memory, Tools, etc.

📌 It's rule-based. No thinking or decisions involved.

🧠 **Example:**

You create a QnA chatbot that always does:

Take input → Ask GPT → Show response
That's a **chain**.

🧠 **What is an Agent?**

Now, imagine your app needs to **think**, make decisions, and choose tools dynamically. That's where **agents** come in.

🧠 **Example Analogy:**

A personal assistant (Agent):

You say: "Book me a flight to Goa under ₹5000"

The assistant:

1. Search flight APIs
2. Chooses the cheapest flight
3. Sends confirmation

Your assistant is **thinking, deciding, and choosing tools**.
That's an **agent**.

🤖 **In LangChain:**

An **agent** is powered by an LLM + a **list of tools** (like a calculator, search, code executor, etc.).

The agent decides *what to do next* — it reasons and acts.

🤔 So what are we building in LangChain?

✅ If you're making a step-by-step AI app, you build a Chain

Example: Chatbot, text summarizer, resume generator

✅ If you want the AI to decide what to do, you use an Agent

Example: AI Assistant, AutoGPT, research bots

🔧 Summary Table:

Concept	Like...	Does it think?	Dynamic?	Use Case Example
Chain	A recipe (fixed steps)	❌ No	❌ No	Ask GPT → Get Answer
Agent	A smart assistant (brain)	✅ Yes	✅ Yes	"Search web → summarize → respond"

🚀 When to Use What?

- Just connect LLM + Prompt → Use Chain
- Need decision-making or tool use → Use Agent

So, can we say this?

"Integrate simple Node.js + ChatGPT API = Creating Chain?"

YES!

When you integrate the OpenAI API with Node.js,
write your own prompt, and
process the output

You're essentially doing what **LangChain Chains** do under the hood.

```
// Basic Node + OpenAI call = Like a simple Chain
const result = await openai.createChatCompletion({
  model: "gpt-3.5-turbo",
  messages: [{ role: "user", content: "Summarize this blog..." }]
});
```

- ✓ This is **manual chaining** — you are building your own logic.
- ✗ But you're not using **LangChain's tools** (PromptTemplate, OutputParser, etc.).

Just like we can build web pages using plain HTML, CSS, and JavaScript, using ReactJS makes the process faster, more efficient, and scalable — especially for building complex interfaces.

AND

Creating Agents is helpful with LangChain because of built-in tools?

YES! 100% true.

Agents are like **intelligent workers** who:

- Use tools (search, calculator, code interpreter)
- Make decisions based on output
- Run multiple steps on their own (without fixed logic)

LangChain provides:

- Built-in **Tool integration** (like Zapier, Google Search, API calls)
- **Memory handling**
- Auto-planning and task execution (like a mini AI assistant)

Without LangChain, building agents from scratch would be a lot of work.

 Summary:

Concept	Need LangChain?	Can You Build Without It?
Basic prompt + response (Chain)	✗ No	✓ Yes, via OpenAI API
Logic flow with multiple prompts	✗ No	✓ Yes, manually
Tools, Memory, Decision Making (Agent)	✓ Yes	✗ Very hard to do manually

How to Build a Basic Chain

Example: "Q&A Bot" using LangChainJS

We'll build:

- Input: "Who is Elon Musk?"
- LLM: Processes it
- Output: Answer with parsing

👉 Steps:

```
npm install langchain @langchain/openai dotenv
```

```
import { ChatOpenAI } from "@langchain/openai";
import { PromptTemplate } from "@langchain/core/prompts";
import { LLMChain } from "langchain/chains";

const model = new ChatOpenAI({ temperature: 0.5 });

const prompt = PromptTemplate.fromTemplate(
  "Answer the following question: {question}"
);

const chain = new LLMChain({ llm: model, prompt });

const response = await chain.invoke({ question: "Who is Elon Musk?" });
console.log(response.text);
```

👉 Output: "Elon Musk is a billionaire entrepreneur..."

🟢 `npm install langchain @langchain/openai dotenv`

This command installs **3 packages**:

✓ 1. **langchain**

📦 The **main LangChain framework** that gives you tools to:

- Build chains
- Work with *prompts*, *memory*, *agents*
- Integrate with various LLMs (OpenAI, Cohere, Ollama, etc.)

It's the **core** of your LangChain-powered app.

✓ 2. **@langchain/openai**

📦 A **LangChain adapter** to connect to **OpenAI models** (like GPT-3.5, GPT-4, etc.).

This package allows you to use:

- **ChatOpenAI** – for chat models
- **OpenAIEmbeddings** – for vector search
- **OpenAI** – for completion models

So yes — you're using **OpenAI's API** as your backend LLM in this setup.

👉 That's why you need to **provide an OpenAI API key** either in code or via **.env**.

✓ 3. **dotenv**

📦 A utility to **load environment variables from a .env file**.

You typically use it like this:

```
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxx
```

And then in your Node.js code:

```
import dotenv from "dotenv";
dotenv.config();
```

This keeps your API keys **secure** and **separate** from your codebase.

🔒 **So, Are We Using the OpenAI API Here?**

✓ Yes — by default, this code uses **OpenAI's API** to access models like GPT-3.5 or GPT-4.

You **must provide** an API key to use OpenAI in this way:

- Either in `.env` using `dotenv`
- Or by setting it manually in the constructor

🚀 Can We Use Ollama Instead of OpenAI?

Yes — LangChain also supports **local models** via Ollama, but you need a different wrapper. Example (to use Ollama with LangChain):

```
import { ChatOllama } from "langchain/chat_models/ollama";
const model = new ChatOllama({ model: "llama2" });
```

This does **not need an API key**, because it's local.

- Now, OPENAI has changed its policy to give free credit for the new accounts
- So we will use Gemini API keys for development

First LangChain-powered AI app with Gemini 2.5 Flash locally using Node.js

```
import {config} from "dotenv";
config();

import { ChatGoogleGenerativeAI } from "@langchain/google-genai";
import { LLMChain } from "langchain/chains";
import { PromptTemplate } from "@langchain/core/prompts";

// 1. Setup the model
const model = new ChatGoogleGenerativeAI({
  model: "models/gemini-2.5-flash", // Free-tier model
  maxOutputTokens: 2048,
  temperature: 0.7,
  apiKey: process.env.GOOGLE_API_KEY,
});

// 2. Create a prompt
```

```
const prompt = PromptTemplate.fromTemplate(
  "Explain the concept of {topic} to a beginner."
);

// 3. Build a chain
const chain = new LLMChain({
  llm: model,
  prompt: prompt,
});

// 4. Run it
const res = await chain.run("quantum computing");
console.log("🧠 Gemini Response:\n", res);
```

🧩 Here's What You've Built:

✅ A **Chain** in LangChain = Prompt Template + Model + Logic

And you're using:

- 📦 [@langchain/google-genai](#): Gemini integration
- 📦 [langchain/chains](#): To run a chain
- 📦 [@langchain/core/prompts](#): For prompt templating
- 🔑 [dotenv](#): To securely manage your Gemini API key

🧠 What to Explain to Students:

Step	What It Does	Why It Matters
✓ Model	Uses Gemini 2.5 Flash (free & fast)	You don't need OpenAI to get started
✓ Prompt	Dynamic input (<code>{topic}</code>)	You can reuse the prompt with any topic
✓ Chain	Combines model + prompt	Think of it as an AI pipeline or mini-app
✓ Run	Gives output for " <code>quantum computing</code> "	Can build tools like tutors, explainers, etc.

What is LCEL?

LCEL stands for **LangChain Expression Language**.

It's a newer, more *composable*, *modular*, and *future-proof* way to build LLM pipelines using `.pipe()` and `.invoke()` instead of older abstractions like `LLMChain`.

```
import { config } from "dotenv";
config();

import { ChatGoogleGenerativeAI } from "@langchain/google-genai";
import { PromptTemplate } from "@langchain/core/prompts";

// 1. Setup the model
const model = new ChatGoogleGenerativeAI({
  model: "models/gemini-2.5-flash", // Free-tier model
  maxOutputTokens: 2048,
  temperature: 0.7,
  apiKey: process.env.GOOGLE_API_KEY,
});

// 2. Create a prompt
const prompt = PromptTemplate.fromTemplate(
  "You are a helpful assistant. Answer the question: {question}"
);

// 3. Create the chain using LCEL
const chain = prompt.pipe(model);

// 4. Run the chain with a user input
const response = await chain.invoke({
  question: "What is the future of AI in healthcare?",
});

console.log("Response from LLM:", response.text);
```

💡 Benefits of LCEL:

1. ✅ **Simple & Readable** (clean chaining style)
2. 🔗 **Composable** – you can chain multiple pieces together (`prompt → model → parser → output`)
3. ⚙️ **Integrates better with tools, agents, and retrievers**
4. 📈 **Optimized performance** for production-ready apps

🔧 Quick Bonus:

If you want to **parse structured output**, you can even do this:

```
import { StringOutputParser } from "@langchain/core/output_parsers";

const outputParser = new StringOutputParser();

const chain = prompt.pipe(model).pipe(outputParser);
```

How does a pipe work?

- The pipe method is part of the LangChain Expression Language (LCEL).
- When you use objects like PromptTemplate or LLMs from LangChain, they come with the pipe method built in.
- This method allows you to chain together different components (like prompts, LLMs, output parsers, etc.) in a modular way.

Why don't you see a pipe in your code?

- You don't need to import or define it yourself.
- It's automatically available on LangChain components (like `PromptTemplate` and `LLMs`) because they inherit from base classes that include pipe.

🧠 What does `.pipe()` actually do?

Think of `.pipe()` like a data pipeline in JavaScript:

```
prompt.pipe(model)
```

This means:

- First, the `prompt` receives an input (e.g., `{ question: "What is AI?" }`)
- It transforms it into a prompt string
- Then `.pipe()` sends this string to the `model`
- The model runs it and returns the final result

Analogy

Imagine a **water filter system**:

1. **PromptTemplate** is the first filter — it formats the input
2. **Model** is the next filter — it processes the formatted question and generates a response
3. **pipe()** is the pipe connecting the filters

Day 6: LangChain + LLM Agents

What Are Agents in LangChain?

Agents are more flexible than chains.

They don't follow a fixed path. Instead, they:

- Think, decide, and act
- Call tools (like Calculator, Google Search, API calls)
- Manage multiple steps

 *Example Use Case:*

```
"Who is India's Prime Minister, and what's the current time in Delhi?"
```

The agent will call:

- LLM → for name
- Tool (clock or API) → for time

Tools an Agent Can Use:

- Web search
- Calculator
- Document reader
- Database query
- Python code execution

LangChain **does not itself** provide actual services like web search engines, databases, or calculators.

Instead, it provides **adapters/wrappers** — reusable **abstractions** — that connect your agent to *external tools or APIs*.

Hands-On: Live News Q&A Bot with LangChain Agent

```
npm install langchain @langchain/google-genai dotenv
```

```
import {config} from "dotenv";
config();

import { ChatGoogleGenerativeAI } from "@langchain/google-genai";
import { initializeAgentExecutorWithOptions } from "langchain/agents";
import { SerpAPI } from "@langchain/community/tools/serpapi";

const model = new ChatGoogleGenerativeAI({
  model: "models/gemini-2.5-flash", // Free-tier model
  maxOutputTokens: 2048,
  temperature: 0.7,
  apiKey: process.env.GOOGLE_API_KEY,
});

// ✅ Directly using built-in tool
const searchTool = new SerpAPI(process.env.SERPAPI_API_KEY, {
  location: "India", // Optional: Set region
});

const agent = await initializeAgentExecutorWithOptions(
  [searchTool],
  model
);

// 🖋️ Try a question
const res = await agent.invoke({
  input: "What is the latest news about ISRO?",
});

console.log("🧠 Final Output:", res.output);
```

When you're building an AI app using **LangChain**, you're actually using a set of **different libraries** that work together.

1. **@langchain/core** — *The Base Tools*

- Think of this like the **foundation or skeleton** of the LangChain framework.
- It includes **basic building blocks** like:
 - how prompts are formatted,
 - how chains are connected,
 - and how LangChain's internal language (called LEL – LangChain Expression Language) works.

💡 Example: If LangChain is a car, **@langchain/core** is the engine and steering system.

2. **@langchain/community** — *Connectors to External Services*

- This is like a **toolbox of integrations**.
- It contains **ready-made tools** to connect with third-party services like:
 - SerpAPI (Google search),
 - Pinecone (vector DB),
 - HuggingFace (models),
 - and many others.

💡 Example: If you want your app to “Google something” or talk to a database, **@langchain/community** gives you that ability.

3. **Partner Packages** — *Dedicated Light-Weight Tools*

- These are **specialized packages** for well-known services like:
 - **@langchain/openai** → for OpenAI (ChatGPT)
 - **@langchain/google-genai** → for Google Gemini
 - **@langchain/anthropic** → for Claude

They only include what's needed to talk to that specific provider and depend on **@langchain/core** only.

💡 Example: If you only want to use OpenAI, you can just use **@langchain/openai** without needing the entire toolbox.

4. langchain — The Smart Brain Stuff

- This package has **prebuilt logic** like:
 - **agents** (like smart assistants that decide what tool to use),
 - **chains** (a sequence of steps like: search → summarize → answer),
 - and **retrievers** (to search your own documents).

💡 Example: If `@langchain/core` is the engine, and `@langchain/community` are the tools, then `langchain` is the **driver** who knows how to use them smartly.

Official documentation for these resources:

Langchain: <https://www.langchain.com/>

JS Langchain Docs: <https://js.langchain.com/docs/introduction/>

Python Langchain Docs: <https://python.langchain.com/docs/introduction/>

V3 Langchain API: <https://v03.api.js.langchain.com/index.html>

Langchain chat: <https://chat.langchain.com>

SerpAPI: <https://serpapi.com/>

Always Search NPM Package Libraries for Langchain:

@langchain/core: <https://www.npmjs.com/package/@langchain/core>

```
npm i @langchain/core
```

Points to check and validate

→ While installing packages, you may see this kind of error

```
npm error code ERESOLVE
npm error ERESOLVE unable to resolve dependency tree
npm error
npm error While resolving: langchain-dev@1.0.0
npm error Found: dotenv@17.2.0
npm error node_modules/dotenv
npm error   dotenv@"^17.2.0" from the root project
npm error
npm error Could not resolve dependency:
npm error Fix the upstream dependency conflict, or retry
npm error this command with --force or --legacy-peer-deps
npm error to accept an incorrect (and potentially broken) dependency
resolution.
```

This is a common issue with npm v7+, which enforces strict peer dependency resolution.

Use **--legacy-peer-deps** (Recommended for most users)

This flag tells npm to ignore peer dependency conflicts and install anyway (like npm v6 used to do):

```
npm install @langchain/community --legacy-peer-deps
```

Agent's Behavior:

- If the user's question **needs external data**, the agent will **invoke the tool**.
- If it can answer from its **own knowledge**, it might **skip using any tool**.

Bonus Point

LangChain gives you more than just "Chains" and "Agents".

Chains and Agents are the **two most important “high-level tools”**, but LangChain provides a **full toolbox** to help you build complete AI-powered apps.

🧠 Here's What LangChain Really Gives You:

✅ 1. LLM Wrappers

- Helps you connect to LLMs like **OpenAI**, **Gemini**, **Anthropic**, **Mistral**, etc.
- You can call models using a common interface.

```
import { ChatOpenAI } from "@langchain/openai";  
const model = new ChatOpenAI({ temperature: 0 });
```

✅ 2. Chains

- Sequence of steps: input → prompt → model → output
- You build simple logic flows with it (e.g. QnA bot).

✅ 3. Agents

- LLMs that can **decide** what tool to use.
- Used when you need dynamic behavior like a smart assistant.

✅ 4. Tools

- 1) Pre-built functions that agents can use like:
 - a) **Calculator**
 - b) **Web search**
 - c) **Python executor**
 - d) **API callers**

You can even build **your own custom tool** and plug it in.

✓ 5. Memory

- Enables the LLM to remember previous messages.
- Used for **chat history**, context retention, and user personalization.

```
import { BufferMemory } from "langchain/memory";
```

✓ 6. Prompts

- You get advanced prompt templating tools.
- Create **dynamic prompts** using variables, inputs, and examples.

```
import { PromptTemplate } from "@langchain/core/prompts";
```

✓ 7. Output Parsers

- Helps convert LLM text into a structured format:
 - JSON
 - Numbers
 - SQL
 - etc.

✓ 8. Document Loaders & Text Splitters

- Useful for RAG and working with large documents.
- Load PDFs, URLs, Word docs, CSVs, etc.
- Split them into chunks for processing.

✓ 9. Vector Store Integration

- Easily connect to Pinecone, Chroma, FAISS, etc.
- Used for **RAG (Retrieval-Augmented Generation)** apps.

✓ 10. Runnables (NEW Core Concept)

- These are **modular building blocks** (like functions or nodes).
- Can be reused, composed, and executed anywhere (chain, agent, etc.).
- This is part of the new **LangChain Expression Language (LCEL)**.

Think of LangChain as:

A **construction toolkit** for LLM apps:

You choose your blocks (LLM, chain, agent, memory, etc.)
and assemble them based on your use case.

 Final Summary:

Feature	Use Case Example
LLM Wrappers	Connect to OpenAI, Gemini, Claude
Chains	QnA Bot, Summarizer, Text Generator
Agents	Smart Assistant, Web Research Bot
Tools	Calculator, Browser, Custom APIs
Memory	Chatbot with context
Prompt Templates	Dynamic and structured prompts
Output Parsers	Convert text into JSON, etc.
Loaders/Splitters	Break documents into chunks for processing
Vector Stores	For document search + RAG
Runnables (LCEL)	Modular, composable, flexible app logic

Day 7: Project: Create Your Own AI Chatbot using Node.js + Google Gemini - “**Smart Restaurant Assistant**”

Build a chatbot using Node.js + Express + Ollama (or Gemini API) that can answer your food-related queries (like "Suggest a breakfast dish", or "Give me a healthy lunch option for today").

What You'll Build (Beginner-Friendly):

A simple AI chatbot that:

- Answers food/meal questions
- Can optionally search the web using SerpAPI if the LLM doesn't know
- Has a basic HTML/CSS frontend to chat with the bot
- Works with **Gemini API** (cloud)

Step-by-Step Breakdown

Step 1: Set up the Project

Create a folder and initialize a project:

```
mkdir smart-restaurant
cd smart-restaurant
npm init -y
```

Install dependencies:

```
npm install express cors body-parser dotenv
npm install @langchain/core @langchain/community
npm install axios
```

If you're using Gemini:

```
npm install @langchain/google-genai
```

✅ Step 2: Basic Express Server

Create `server.js`:

```
const express = require("express");
const bodyParser = require("body-parser");
const cors = require("cors");
require("dotenv").config();

const app = express();
app.use(cors());
app.use(bodyParser.json());

app.listen(3000, () => {
  console.log("Server running on http://localhost:3000");
});
```

✅ Step 3: Setup LLM

For Gemini (Google):

```
const model = new ChatGoogleGenerativeAI({
  model: "models/gemini-2.5-flash", // Free-tier model - use pro
  maxOutputTokens: 2048,
  temperature: 0.7,
  apiKey: process.env.GOOGLE_API_KEY,
});
```

✓ Step 4: 🍲 Tool: Restaurant Menu Tool

```
const getMenuTool = new DynamicStructuredTool({
  name: "getMenu",
  description: "Returns the final answer for today's menu for the given category (breakfast, lunch, or dinner). Use this tool to answer the user's menu question directly.",
  schema: z.object({
    category: z.string().describe("Type of food. Example: breakfast, lunch, dinner"),
  }),
  func: async ({ category }) => {
    const menus = {
      breakfast: "Aloo Paratha, Poha, Masala Chai",
      lunch: "Paneer Butter Masala, Dal Fry, Jeera Rice, Roti",
      dinner: "Veg Biryani, Raita, Salad, Gulab Jamun",
    };
    return menus[category.toLowerCase()] || "No menu found for that category.";
  },
});
```

We are creating a **tool** that your AI chatbot (LangChain Agent) can use to answer a user's question like: "What's for lunch?"

This tool is called: `getMenuTool`

🧱 What is `DynamicStructuredTool`?

Think of `DynamicStructuredTool` as a **custom tool** you give to the chatbot. It works like this:

🧠 "Hey Bot! If someone asks about the food menu, use this tool to get the correct answer."

LangChain lets you add tools like:

- Calculators
- Web search
- Database queries
- And custom ones like this: **getMenuTool**

Let's Break It Down

```
const getMenuTool = new DynamicStructuredTool({
```

You are creating a new tool called getMenuTool.

```
name: "getMenu",
```

This is the **name of the tool** the AI uses internally. It's like giving your tool a label or ID.

```
description: "Returns the final answer for today's menu..."
```

This **tells the AI** when to use this tool. It says:

"If someone asks about today's menu — breakfast, lunch, or dinner — use this tool."

```
schema: z.object({  
  category: z.string().describe("Type of food...")  
})
```

This defines the **input** that the tool needs.

You're saying:

- The tool takes **one input**: a string called **category**
- This string can be: "**breakfast**", "**lunch**", or "**dinner**"

You're using **zod** (a schema validation library) to define what kind of input is expected.

```
func: async ({ category }) => {
  const menus = {
    breakfast: "...",
    lunch: "...",
    dinner: "...",
  };
  return menus[category.toLowerCase()] || "No menu found for that category.";
}
```

This is the actual function that runs when the AI uses the tool.

- It checks the `category` (like `lunch`)
- Converts it to lowercase (to avoid case errors)
- Returns the correct menu from the list

If the user types something wrong (like `brunch`), it returns:

`"No menu found for that category."`

Final Thought: What We Created?

You created a **custom tool** named `getMenuTool`, which:

Feature	Description
Tool Name	<code>getMenu</code>
Input	One word (category): <code>"breakfast"</code> , <code>"lunch"</code> , or <code>"dinner"</code>
Output	The actual menu as a string
Purpose	Help the AI answer menu-related questions
Tech used	<code>DynamicStructuredTool</code> + <code>zod</code> schema

Now, for creating Agents, we are using **createToolCallingAgent**

vs Why use createToolCallingAgent over initializeAgentExecutorWithOptions?

✓ **createToolCallingAgent** (New Way)

- More **modern, flexible, and robust**
- Gives **full control over the prompt**
- Supports **structured tools**, better at understanding **schemas**
- Works well with **LangChain v0.1+**

Now, to work with createToolCallingAgent, we need to create a Prompt and an Agent

♦ Step-by-Step Explanation

```
const prompt = ChatPromptTemplate.fromMessages([
  ["system", "You are a helpful assistant that uses tools when needed."],
  ["human", "{input}"],
  ["ai", "{agent_scratchpad}"]
]);
```

🔍 What's happening here?

You're **defining the conversation pattern** between:

- System (the behavior guide for the AI)
- Human (user input)
- AI (thinking, tool-using, and replying)

Let's break it down:

- **"system"** → Tells the AI: "Behave like a smart assistant. Use tools when necessary."
- **"human"** → This is where the user's actual question will be inserted (**{input}**)
- **"ai"** → This is like a scratchpad [*piece of paper*]. When the AI is trying to **reason out or decide to use a tool**, it will think here.

```
const agent = await createToolCallingAgent({
  llm: model,
  tools: [getMenuTool],
  prompt
});
```

What is this doing?

This is where you **create the actual agent** (AI brain that can think and use tools):

- `llm: model` → This is your language model (like `ChatOpenAI`, `Ollama`, `Gemini`, etc.)
- `tools: [getMenuTool]` → This gives the agent access to your function (the tool to get menu)
- `prompt` → You're plugging in the custom prompt we just defined

So now this **AI agent is ready to receive questions and decide**:

- Can I answer it myself?
- Should I call the tool?
- If yes, pass the correct input to the tool
- Return the final answer to the user

```
const executor = await AgentExecutor.fromAgentAndTools({
  agent,
  tools: [getMenuTool],
  maxIterations: 10, // Optional: Let it think more steps if needed
});
```

⚙️ What's happening here?

Now you're creating a **runner/executor** that:

- Takes the agent (you just built)
- Executes it with tool access
- Limits the number of thinking steps (**maxIterations**)

🔄 Why **maxIterations: 10**?

Sometimes the agent needs multiple steps to solve a problem. You allow it to reason up to 10 steps.

Now, create a front-end part for our application

```
<!-- public/index.html -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Restaurant Chatbot</title>
  <style>
    body { font-family: sans-serif; max-width: 600px; margin: auto;
padding: 2rem; }
    input, button { padding: 0.5rem; margin-top: 1rem; width: 100%; }
    #chat { white-space: pre-wrap; margin-top: 1rem; }
  </style>
</head>
<body>
  <h2>🍴 Restaurant AI Assistant</h2>
  <div id="chat"></div>
  <input type="text" id="userInput" placeholder="Ask me: What's on the
dinner menu?" />
  <button onclick="sendMessage()">Send</button>

  <script>
    async function sendMessage() {
      const input = document.getElementById("userInput").value;
      const res = await fetch("/api/chat", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ input })
      });
      const data = await res.json();
      const chatDiv = document.getElementById("chat");
      chatDiv.innerHTML += `👤 You: ${input}\n🤖 Bot:
${data.output}\n\n`;
      document.getElementById("userInput").value = "";
    }
  </script>
</body>
</html>
```

✓ What We're Building:

A **very simple web page** where:

- You (the user) type a message.
- That message is sent to the **backend** (`/api/chat`) where our **LangChain agent** lives.
- The bot replies with an answer (like today's dinner menu).
- You see the **chat conversation** update in real-time.

`<h2>` is the chatbot title.

`<div id="chat">`: where messages are shown.

`<input>`: lets you type your message.

`<button>`: when clicked, it runs `sendMessage()` function.

The JavaScript Part 🧠

```
const input = document.getElementById("userInput").value;
```

Grabs the text you typed in the input box.

```
const res = await fetch("/api/chat", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ input })
});
```

Sends your input to the backend using a **POST request** to `/api/chat`.

`{ input }` sends the message as a JSON object like `{ "input": "lunch" }`

```
const data = await res.json();
```

Waits for the server to send back a response.

Converts it to a usable JavaScript object.

```
const chatDiv = document.getElementById("chat");
chatDiv.innerHTML += `👤 You: ${input}\n🤖 Bot: ${data.output}\n\n`;
```

Adds your question and the bot's reply to the chat box.

```
document.getElementById("userInput").value = "";
```

Clears the input box so you can type the next message.

✅ Final Output:

```
👤 You: What's for lunch?
🤖 Bot: Paneer Butter Masala, Dal Fry, Jeera Rice, Roti
```

🔄 How It All Works Together:

1. You type something → hit "Send".
2. It goes to your **Node.js + LangChain backend** (at `/api/chat`).
3. The **LangChain agent** decides how to respond.
4. You see the answer right on the page.

```
const __dirname = path.resolve();
```

✅ What is `__dirname`?

In Node.js, `__dirname` usually refers to the **directory where your current file is located**.

But in **ES Modules** (like when we use `"type": "module"` in `package.json`), the `__dirname` variable is **not available by default**.

So we use:

```
path.resolve()
```

This gives us the **absolute path** of the current working directory, which we store in a variable named `__dirname` (just like the original Node.js behavior).

- ♦ So now `__dirname` holds the full path to your project folder like:

```
/Users/prince/langchain-dev/smart-restaurant-assistant
```

- ♦ `app.get("/", (req, res) => { ... });`

This line defines a **GET route** in Express.

✓ **What does it do?**

- It means: “When a user opens the website and hits the root URL `/` (i.e., homepage), do the following...”

req and res

- `req` = the incoming **request** from the browser
- `res` = the outgoing **response** we send back

```
res.sendFile(...)
```

This tells Express: “Send this **file** as the response.”

This is how we **build the full path** to the file we want to send.

It joins:

- `__dirname` → `/Users/prince/langchain-dev/...`
- `"public"` → the folder where the frontend lives
- `"index.html"` → the actual HTML file

```

app.post("/api/chat", async (req, res) => {
  const userInput = req.body.input;
  console.log('userInput : ', userInput);
  try {
    const response = await executor.invoke({ input: userInput });

    console.log('Response : ', response);
    res.json({ output: response.output });
  } catch (err) {
    console.error(err);
    res.status(500).json({ output: "Sorry, something went wrong. Please try again." });
  }
});

```

This is your **backend API** where your frontend sends the **user's message** (like "Show me the breakfast menu"), and it responds with the **AI's answer** using the LangChain agent.

```

app.post("/api/chat", async (req, res) => {

```

- This defines a **POST** route at `/api/chat`
- **POST** means the frontend is **sending data to the backend** (user input in our case)

```

const userInput = req.body.input;

```

- This line **extracts the user message** sent from the frontend
- In your HTML + JS, you're doing a `fetch("/api/chat", { method: "POST", body: JSON.stringify({ input: ... }) })`
- So here, we extract that `input`

```
const response = await executor.invoke({ input: userInput });
```

✨ This is the **core logic**.

- **executor** is your **LangChain AgentExecutor**.
- It takes the user input and **processes it using LLM + tool (like getMenuTool)**.
- It returns the **AI response**.

Common Errors which you can see while development:

1. Agent stopped due to max iterations

You're encountering the classic "*Agent stopped due to max iterations*" error in LangChain. This means the agent is looping—repeatedly calling the tool without ever producing a final answer. This is a **common pain point** with **tool-calling agents**, especially with Gemini and similar LLMs.

Why does this happen?

- The LLM doesn't realize it should stop after calling the tool and keeps trying to call it again.
- The agent expects the LLM to "wrap up" after a tool call, but the LLM keeps asking for more tool calls.
- This is especially common with custom tools and some LLMs that don't natively support tool-calling.

1. `verbose: true`

- When you set `verbose` to `true`, the agent (or executor) prints detailed logs and information about what it's doing to the console.
- This helps you **see the step-by-step process** — like what tool it calls, what input it sends, what output it gets, what decisions it makes, etc.
- It's very useful for **debugging** or understanding the agent's internal workings.

Example:

Without `verbose`, you might just see the final answer.

With `verbose: true`, you see the entire conversation between agent and tools.

2. `returnIntermediateSteps: true`

- This option tells the agent or executor to **return all the intermediate actions, tool calls, and their results as part of the response**.
- So, instead of just the final output, you get a detailed breakdown of **each step the agent took internally**.
- This is useful if you want to **trace or analyze how the agent arrived at the answer** or debug why it may have looped or made certain decisions.

Example:

If `returnIntermediateSteps` is `true`, your response might include an array like:

```
[
  {
    "action": { "tool": "getMenu", "toolInput": { "category": "dinner" } },
    "observation": "Veg Biryani, Raita, Salad, Gulab Jamun"
  },
  {
    "action": {...},
    "observation": "..."
  }
]
```

Parameter	What it does	Why it's useful
<code>verbose: true</code>	Logs detailed process info to console	Helps you debug and understand steps
<code>returnIntermediateSteps: true</code>	Returns all internal steps in response	Lets you see how the agent reasoned step-by-step

List of **beginner-friendly, ready-to-use tools** from [@langchain/community/tools](https://github.com/langchain-ai/langchain/tree/master/community/tools) you can explore:

✓ 1. Calculator Tool

Package: [langchain/tools/calculator](https://github.com/langchain-ai/langchain/tree/master/community/tools/calculator)

Use Case: When the question involves basic math (e.g., “What’s 32% of 450?”)

Why useful? LLMs aren’t perfect at math. The calculator tool ensures accurate answers.

✓ 2. Google Maps Tool

Use Case: Searching for places, directions, and nearby locations

➡ (Needs Google Maps API Key)

✓ 3. Wikipedia Tool

Package: [@langchain/community/tools/wikipedia](https://github.com/langchain-ai/langchain/tree/master/community/tools/wikipedia)

Use Case: For general factual queries.

E.g., “Who discovered penicillin?”

(No API key needed – works out of the box!)

✓ 4. OpenWeather Tool

Use Case: Live weather info.

E.g., “What’s the weather in Ranchi today?”

➡ Requires OpenWeatherMap API key.

✓ 5. NewsAPI Tool

Use Case: Alternative to SerpAPI for Real News Headlines.

➡ Needs API key from newsapi.org

✓ 6. YouTube Search Tool

Use Case: Get the latest YouTube videos about a topic

➡ Requires YouTube Data API key.

✓ 7. Tmdb Tool (The Movie Database)

Use Case: Get movie info, ratings, and release dates

➡ Needs TMDB API key

✓ 8. Wikipedia + Calculator Combo Agent

Mix Wikipedia and a Calculator to answer:

“How old was Newton when he discovered gravity?”

LARGE LANGUAGE MODELS (LLMs)

CHEAT SHEET



1

Introduction to LLMs

Definition:

LLMs are AI systems that generate human-like text by learning from extensive datasets, used for tasks like translation and conversation.

Popular LLMs:

Notable models include GPT-3 (OpenAI), BERT (Google), and Mistral (Mistral AI).



BERT



Generative AI



FOLLOW US ON LINKEDIN FOR MORE USEFUL CONTENT ABOUT AI!

Core Concepts



2

Transformer Architecture:

Utilizes attention mechanisms to process text, allowing the model to focus on various input parts.

Tokenization:

Breaks text into tokens (words or subwords) for model processing.

Input Representations:

Transforms text into vectorized formats, like word or subword embeddings, to capture semantic meaning and context.

Attention Mechanisms:

Compute relationships between input tokens to highlight relevant information and capture long-range dependencies in text.



3

Transfer Learning and Fine-tuning

Transfer Learning:

Focuses on transferring general knowledge from one domain to another. It often involves using the same objective function and may freeze certain layers to retain general features.

Fine-Tuning:

Goes a step further by specializing the model to a particular task. This may involve modifying the objective function, adjusting specific layers, and unfreezing parts of the model for targeted training.

Types of LLMs

4

LLMs can be broadly categorized into encoder-only models, such as BERT, which are optimized for understanding and processing input data, and decoder-only models, like GPT, which are designed for generating text outputs. There are also encoder-decoder models, such as T5, which integrate both functionalities to handle tasks involving both input understanding and output generation, making them versatile for translation and summarization.



5

Preprocessing Techniques

Text Normalization:

Standardizes text by adjusting casing and removing punctuation.

Stop Words:

Commonly removed words to focus on meaningful content.

Lemmatization and Stemming:

Reduce words to their root form to enhance model efficiency.



6

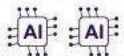
Applications of LLMs

LLMs are rapidly transforming various industries, with projections estimating a \$1.3 trillion market by 2032 for generative AI solutions like ChatGPT, Google Bard, and Bing Chat. They also enhance search functionalities, automate customer support through virtual assistants, detect cyber threats, assist in code development, transcribe audio to text, conduct market research, facilitate keyword analysis, streamline sales processes, and perform sentiment analysis, making them invaluable tools for improving efficiency and gaining insights across various domains.



AI TERMS CHEAT SHEET

AI



Intelligence exhibited by machines, like learning and problem solving.

ML



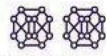
AI that learns from data to improve performance on a specific task.

Deep Learning



A powerful type of ML using complex artificial neural networks.

Neural Network



A network of interconnected nodes inspired by the human brain.

Supervised Learning



ML where training data includes the desired output.

Unsupervised Learning



ML where the algorithm finds patterns in unlabeled data.

Reinforcement Learning



ML where the algorithm learns through trial and error with rewards.

NLP (Natural Language Processing)



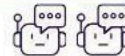
A field of AI focused on how computers can understand and process human language.

Computer Vision



A field of AI focused on enabling computers to see and interpret the visual world.

Chatbot



A conversational AI program designed to simulate conversation with humans.

IoT



Interconnected devices collecting and sharing data.

Cloud Computing



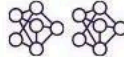
Delivering computing services like storage and servers over the internet.

Bias in AI



Prejudiced outcomes in AI systems due to data or algorithms.

Algorithm



A set of instructions for a computer to follow to solve a problem.

Data Mining



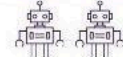
Extracting knowledge and insights from large datasets.

Big Data



Extremely large and complex datasets that are difficult to process with traditional methods.

Robotics



The field of engineering and science focused on designing, building, and operating robots.

Algorithmic Fairness



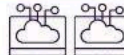
Ensuring AI algorithms are unbiased and avoid discrimination.

Transfer Learning



Using a pre-trained model for a new task, improving efficiency.

Edge Computing



Processing data closer to where it's generated, on devices or local servers.

Explainable AI



Making AI models understandable by humans.

GANs



Generative Adversarial Networks that can generate realistic data like images or text.

Edge AI



AI models specifically designed to run on resource-constrained devices at the network edge.

AI Ethics



The branch of ethics focused on the development and use of AI in a responsible way.

Generative AI (GenAI)



Machine-learning based AI that trains on sets of real-world data most commonly images and text.

Large language model (LLM)



An AI program with a mathematical map across a very large number of dimensions.

Training Data



The data used to teach an AI model how to perform a task.

Fine-tuning LLM



Adjusting a pre-trained LLM for a specific task.

Hallucinations



AI outputs that are factually incorrect or nonsensical.

(GPT)



A particular kind of LLM design, introduced by Open AI, that uses a hybrid training approach.

Prompt



An instruction or question given to an LLM to guide its response.

Prompt Engineering



Crafting effective prompts to get the desired output from an LLM.

Explainable AI



Using a pre-trained model for a new task, improving efficiency.

Neural Network



A network of interconnected nodes inspired by the human brain.

Semantic Database



Stores data with meaning and relationships between concepts.

Structured Data



Data organised in a predefined format, like tables or spreadsheets.

Subscribe to Mindstream for free resources | www.mindstream.news



GenAI Terminology Cheatsheet

GenAI terms you must know!

AGI



Artificial General Intelligence aims to create machines that can learn and reason like humans across various tasks.

ASI



Artificial Super Intelligence is a theory notion of AI that exceeds human intelligence in every capability.

Tokenization



Process of breaking down text into smaller units (tokens), generally used as input for LLMs.

ANI



Artificial Narrow Intelligence specializes in performing a specific set of tasks, lacking the general intelligence of humans.

Attention



Popular mechanism in neural networks that allows focusing on specific parts of the input data.

RAG



Retrieval-Augmented Generation. Appending retrieved information to improve model response.

Fine-Tuning



Adapting an LLM to a specific task or domain by further training it on task-specific data.

Diffusion



Models that generate data by iteratively refining a noise signal, often used in image generation.

Prompting



Providing carefully crafted inputs to an LLM to generate desired outputs.



LLM Cheat sheet – 12 Terms you should know



@rakeshgohel01

1

Prompt

The initial input given to an LLM to generate a response. It can be text, audio or even video

2

Token

Smallest unit of text that an LLM processes. Can be a word, part of word or character

3

Context-Window

The amount of text an LLM can process at once to generate a response

4

Shot-Learning

How much prompt does LLM need to learn a new task. Zero-shot, Few-shot or N-shot

5

Temperature

A parameter that controls the randomness of the model's output

6

Transformers

Self-attention architecture best used to understand relationship between words

7

Fine-tuning

Adjusting a model for a specific task by training it on a small task-specific dataset

8

LoRA

Fine-tune LLMs efficiently by updating only a small number of parameters

9

MoE

Utilizing smaller expert models to train the main LLM for better performance

10

RLHF

Model is trained to improve its responses based on feedback from human evaluators

11

Inference

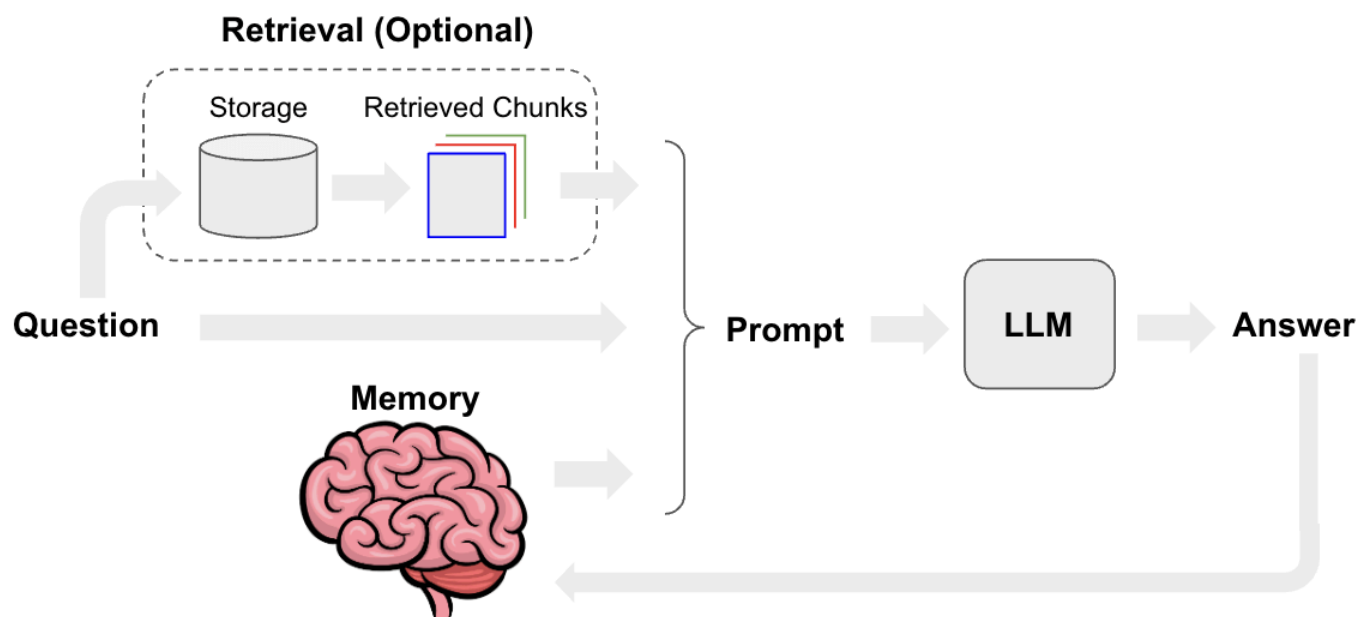
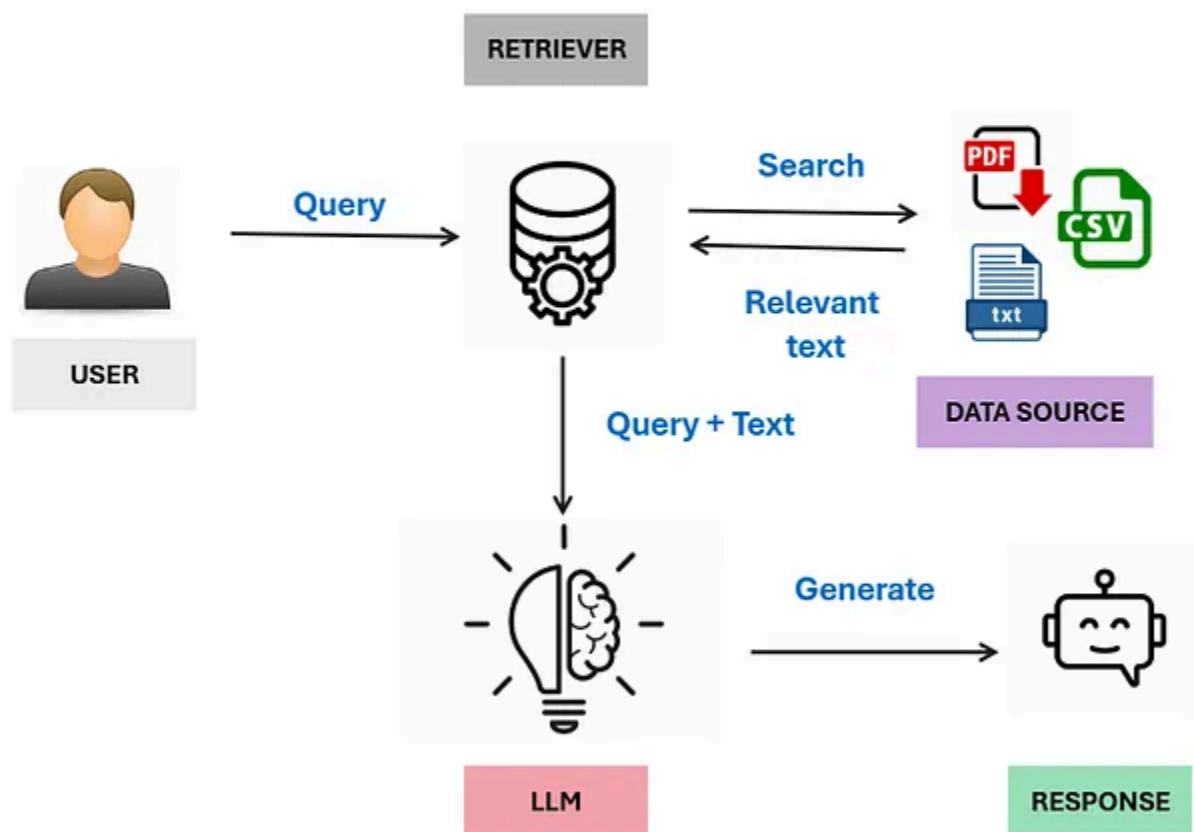
The process of generating outputs based on input data using a trained model

12

Hallucination

When an LLM generates outputs that are factually irrelevant to the input

Retrieval Augmented Generation



LLM TOOLS CHEAT SHEET

in Ganesh Lohar
@ganeshml15



Used for building (training, fine-tuning, and deploying) Large Language Models and NLP applications.

Orchestration of AI agents, task delegation and autonomous decision-making

LLM

Anthropic claude



Deepseek



Mistral



OpenAI



Google Gemini



Meta LLAMA



xAI



General-Purpose AI/ML Frameworks

Build AI apps without writing code, often using visual interfaces

No-code/ Low-code AI Platforms



Flowise



N8N



Langflow



RelevanceAI



Cursor



Lovable



Replit



Chatbase



v0

Tool for evaluating LLM responses, performance, safety, detection

LLM Application Frameworks



Langchain



LlamaIndex



Haystack



OpenLLM



Kernel

For generating Content(text, image, code, videos)

Generative AI/Text to Image



Midjourney



DALL-E



LeonardoAI



GEMINI



Krea.AI

Pretrained models released under open licenses and custom use

Vector Databases



FAISS



ChromaDB



Pinecone



Redis



Weaviate



CrewAI



AutoGen



MetaGPT



Manus.im



Camel AI



Superagent

AI Agent & Multi-Agent Systems

The ChatGPT Cheat Sheet



Act as a [ROLE]

1. Analyst
2. Teacher
3. Marketer
4. Advertiser
5. Mindset Coach
6. Therapist
7. Journalist
8. Inventor
9. Lawyer
10. Ghostwriter
11. Website Designer
12. Best Selling Author
13. Chief Financial Officer
14. Expert Copywriter
15. Prompt Engineer
16. Accountant
17. Project Manager
18. Sports Coach
19. Financial Analyst
20. Full Stack Developer
21. Linux Terminal
22. "Position" Interviewer
23. Excel Sheet
24. Plagiarism Checker
25. Relationship Coach
26. Recruiter
27. JavaScript Console

Create a [TASK]

1. Headline
2. Article
3. Essay
4. Book Outline
5. Email Sequence
6. Social Media Post
7. Product Description
8. Cover Letter
9. Blog Post
10. SEO Keywords
11. Summary
12. Video Script
13. Recipe
14. Sales Copy
15. Analysis
16. Ad Copy
17. Web Page

Show as [FORMAT]

1. A Table
2. A List
3. Summary
4. HTML
5. Code
6. Spreadsheet
7. Graphs
8. CSV file
9. Plain Text file
10. JSON
11. Rich Text
12. PDF
13. XML
14. Markdown
15. Gantt Chart
16. Word Cloud
17. Emojis
18. An Analogy
19. Bullet Points

Set restrictions

1. Use poetic language
2. Adopt a formal tone
3. Write short sentences
4. Code only in HTML/CSS
5. Use Shakespearean style
6. Write using basic English
7. Use only scientific sources
8. Add pop culture references

Prompts For Biz Owners

1. Give me a list of inexpensive ideas on how to promote my business better.
2. Acting as a Business Consultant, What is the best way to solve this problem of [Problem]?
3. Create a 30-Day Social Media Content Strategy based on [Topic 1] & [Topic 2].

Prompts for Developers

1. Develop an architecture and code for a <description> website with JavaScript.
2. Help me find mistakes in the following code <paste code below>
3. I want to implement a sticky header on my website. Can you provide an example using CSS and JavaScript?
4. Please continue writing this code for JavaScript <past code below>

Prompts for Marketers

1. Can you provide me with some ideas for blog posts about [topic]?
2. Write a product description for my [product or service or company].
3. Suggest inexpensive ways I can promote my [company] without using social media.
4. How can I obtain high-quality backlinks to improve the SEO of [website name]?

Prompt Priming

1. ZERO - "Write me 5 Headlines about [Topic]"
2. SINGLE - "Write me 5 Headlines about [Topic]. Here is an example of one headline: 5 Ways to Lose Weight"
3. MULTIPLE - "Write me 5 Headlines about [Topic]. Here is an example of some headlines: 5 Ways to Lose Weight, How to Lose More Fat in 4 Weeks, Say Goodbye to Stubborn Fat, Find a faster way to Lose Weight Fast"

Prompts for Designers

1. Generate examples of UI design requirements for a [mobile app].
2. How can I design a [law firm website] in a way that conveys [trust and authority]?
3. What are some micro-interactions to consider when designing a Fintech app?
4. Create a text-based Excel sheet to input your copy suggestions. Assume you have 3 members in your UX writing team.

C.R.E.A.T.E Formula for Prompt Engineering

1. **C: Character** - Define the AI's role. E.g., "You are a seasoned copywriter with 20 years of experience in successful sales copy."
2. **R: Request** - Be specific with what you need. Instead of "Write a sales email for a sports car," specify, "Write a compelling email for the Cougar Hyper Sport, an electric car with top-tier acceleration."
3. **E: Examples** - Optionally, provide samples for more precise results. Headlines or tone examples can guide the AI's style.
4. **A: Adjustments** - Refine the prompt if it's not perfect. Use instructions like, "Avoid bullet points; use subheads."
5. **T: Type of Output** - Describe the desired format. E.g., "Produce a 500-word article with a title and conclusion."
6. **E: Extras** - Incorporate unique instructions.
 - Ex: "Ignore previous conversations."
 - Ex: "Ask questions before answering."
 - Ex: "Explain your thinking."
 - Ex: "Summarize what you know about me in the first person."
 - Ex: "Use data up to 2023."
 - Ex: "Only use reliable sources and cite them."
 - Ex: "Use CAPS LOCK for emphasis."

Using the C.R.E.A.T.E formula ensures better results, although it may require more time for prompt creation.

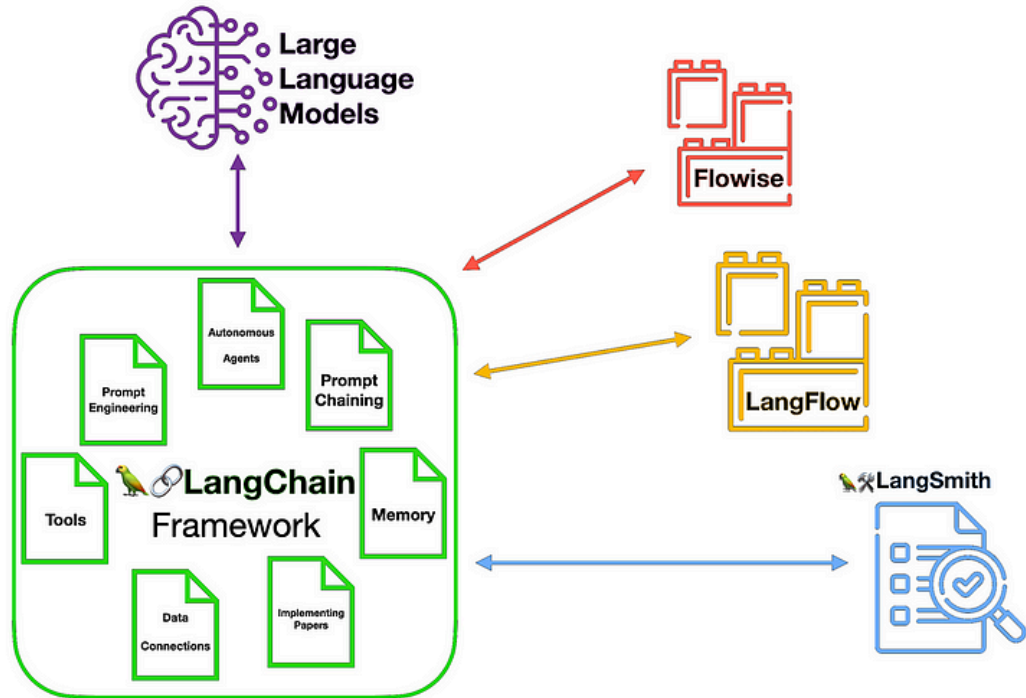
The ChatGPT Cheat Sheet



www.aifire.co



LangChain Ecosystem



www.cobusgreyling.com



14 AI TOOLS TO 10X YOUR PRODUCTIVITY



AI YOUTUBE
SUMMARIES



ICONS

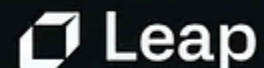


IconGeneratorAi

AI CONTENT



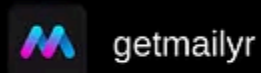
AI IMAGES



AI MEETING



EMAIL



AI SALES



IMAGE



AI AUDIO



CODING



AI
COPYWRITER



PARAPHRASE



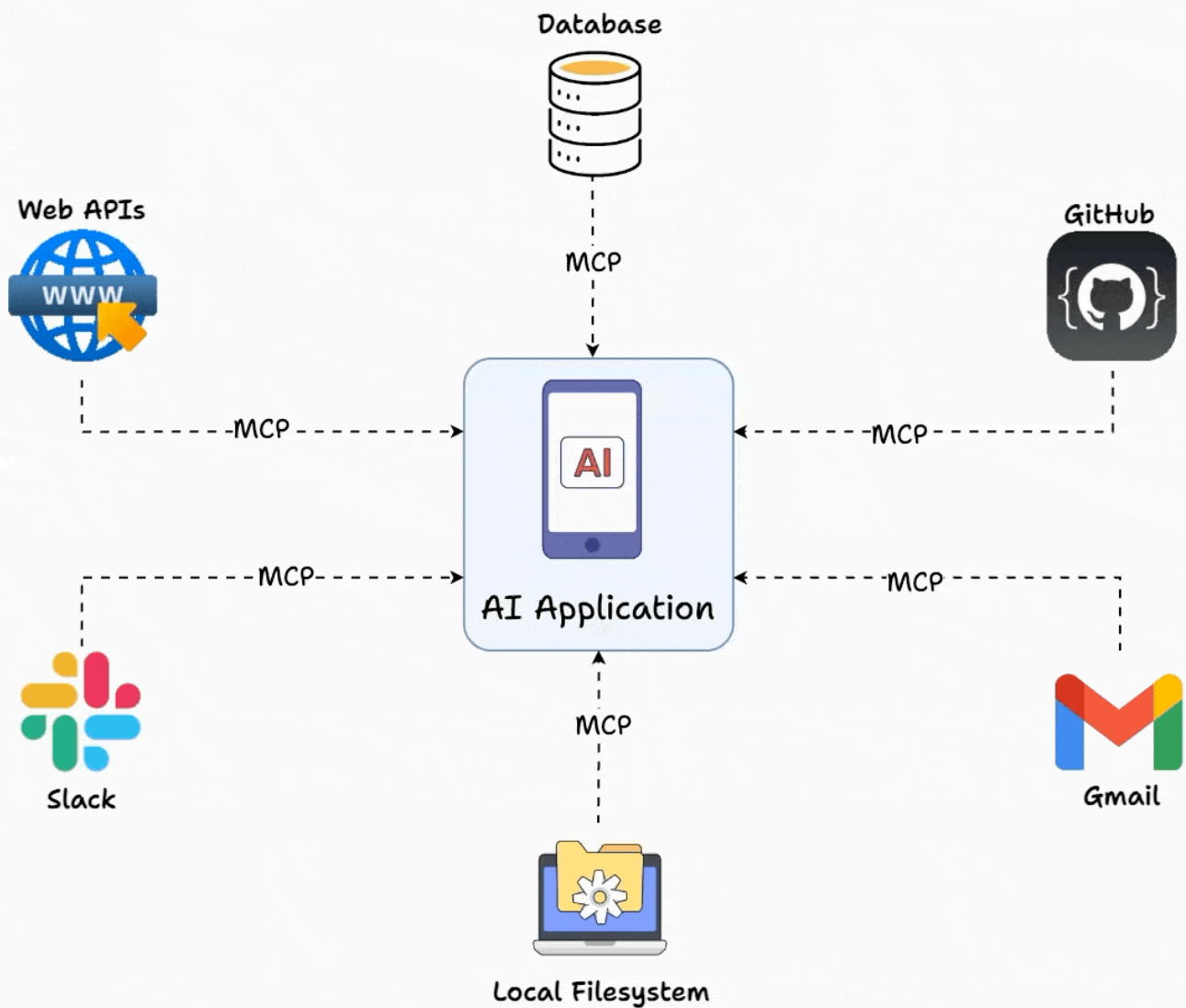
WEBSITE



PROMPTS

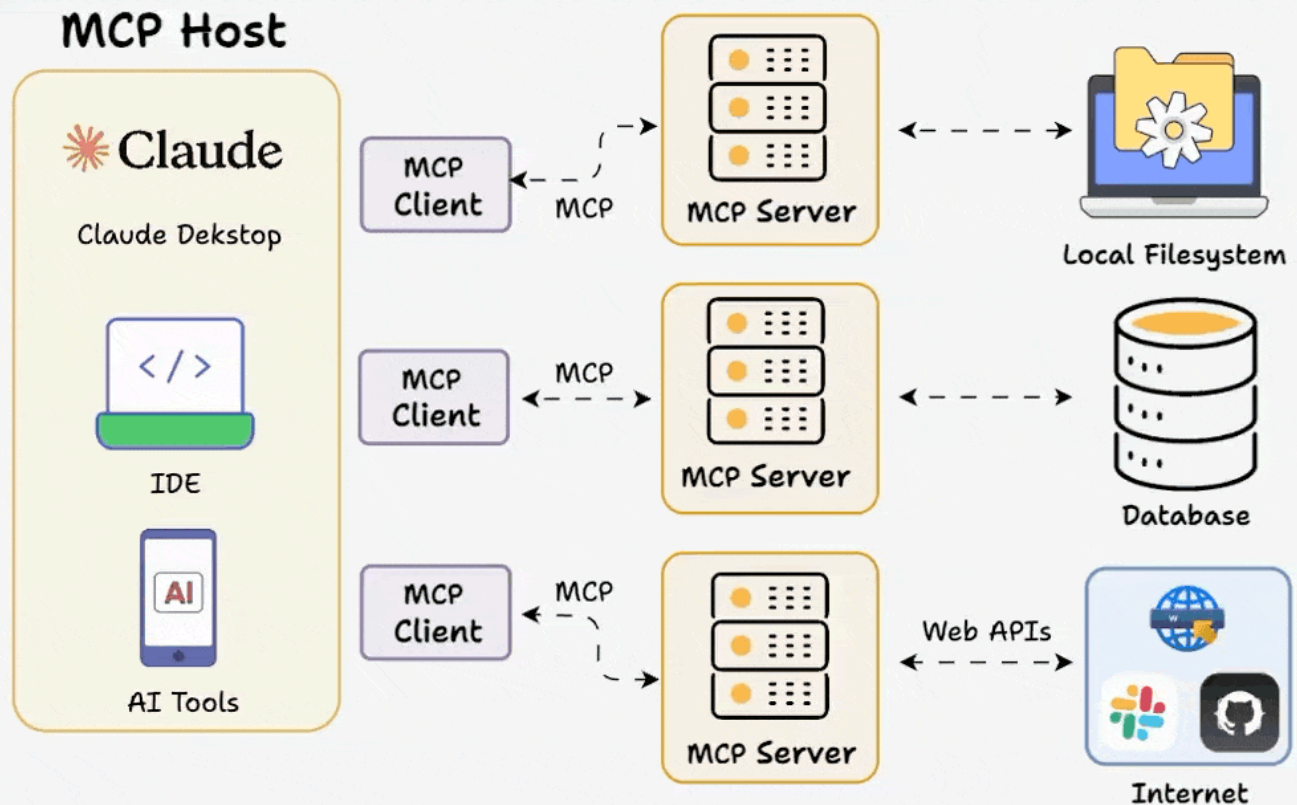


What is MCP ?



Model Context Protocol (MCP), clearly explained

MCP Host



Key Components

